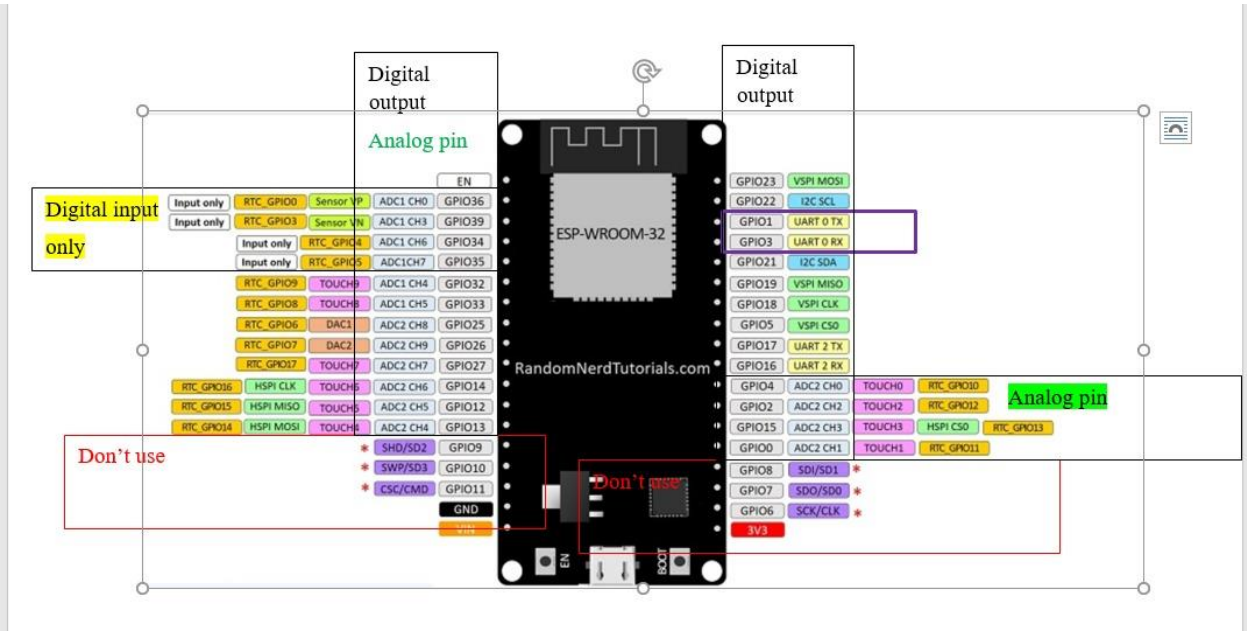
Lab Sheet 04

Title: Interfacing Input & Output Devices with ESP32 in Wokwi

Aims: Learn how to correctly connect common Input Devices and Output Devices to the ESP32 Devkit in the Wokwi online simulator. This lab sheet focuses only on hardware connections (wiring) and includes a brief explanation of each device's purpose.

Tasks:

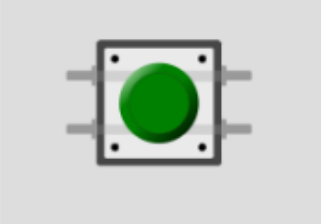
- Open wokwi and start a new project with ESP32 DevKit.
- Use the + button to search and add each component.
- Use different colored wires (Red = Power, Black = GND, others for signals).
- ESP32 GPIO pins are 3.3V logic. Always connect GND properly..



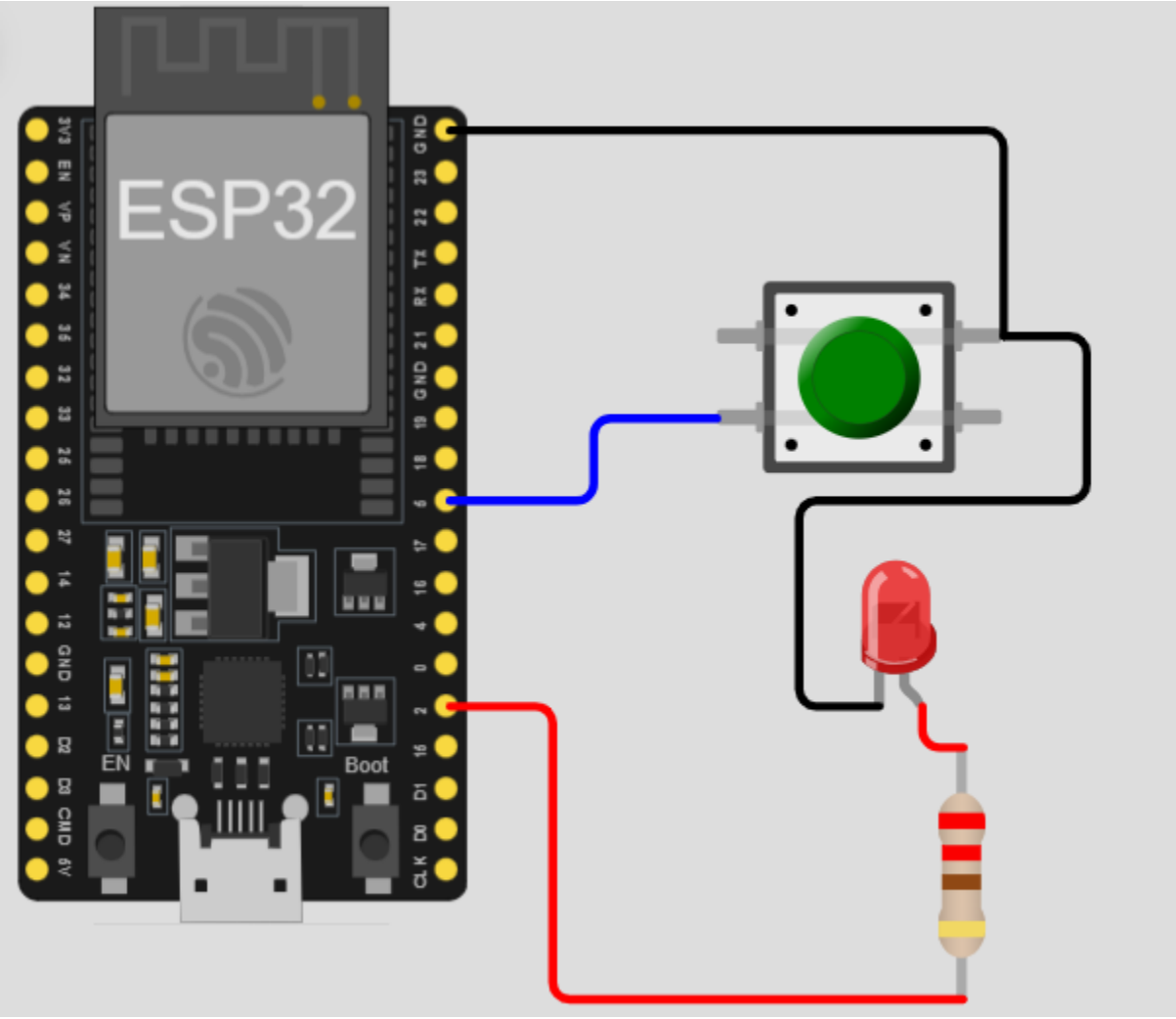
Input Devices

1. Pushbutton (Tactile Switch)

A momentary switch used to detect manual user input (press). Commonly used for triggering actions, resetting systems, or simple user interfaces.

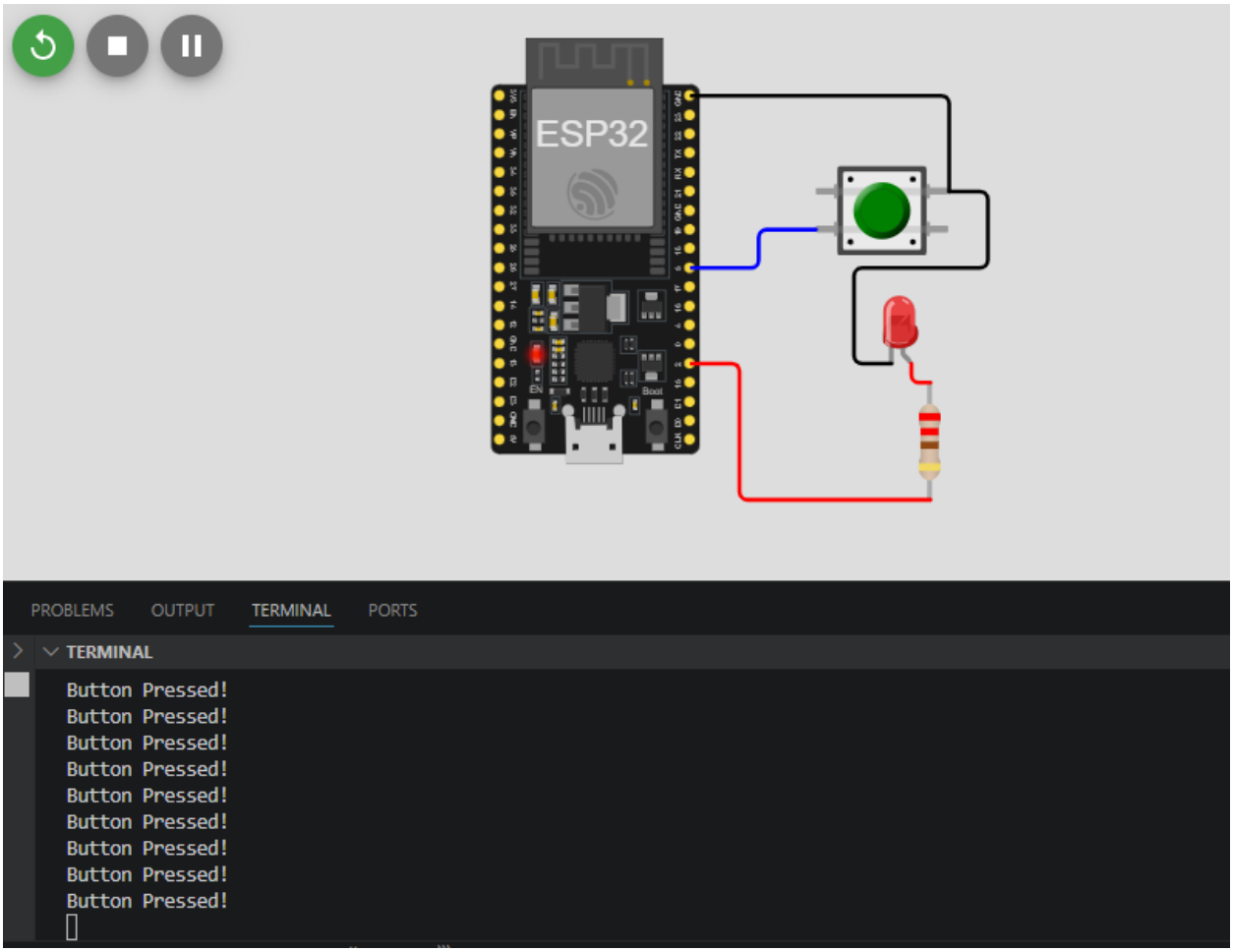


Pins of device	Pins of ESP32
One leg (1.r)	GND
Other leg (2.r)	GPIO 5




```
#include <Arduino.h>
// Define the pins
```

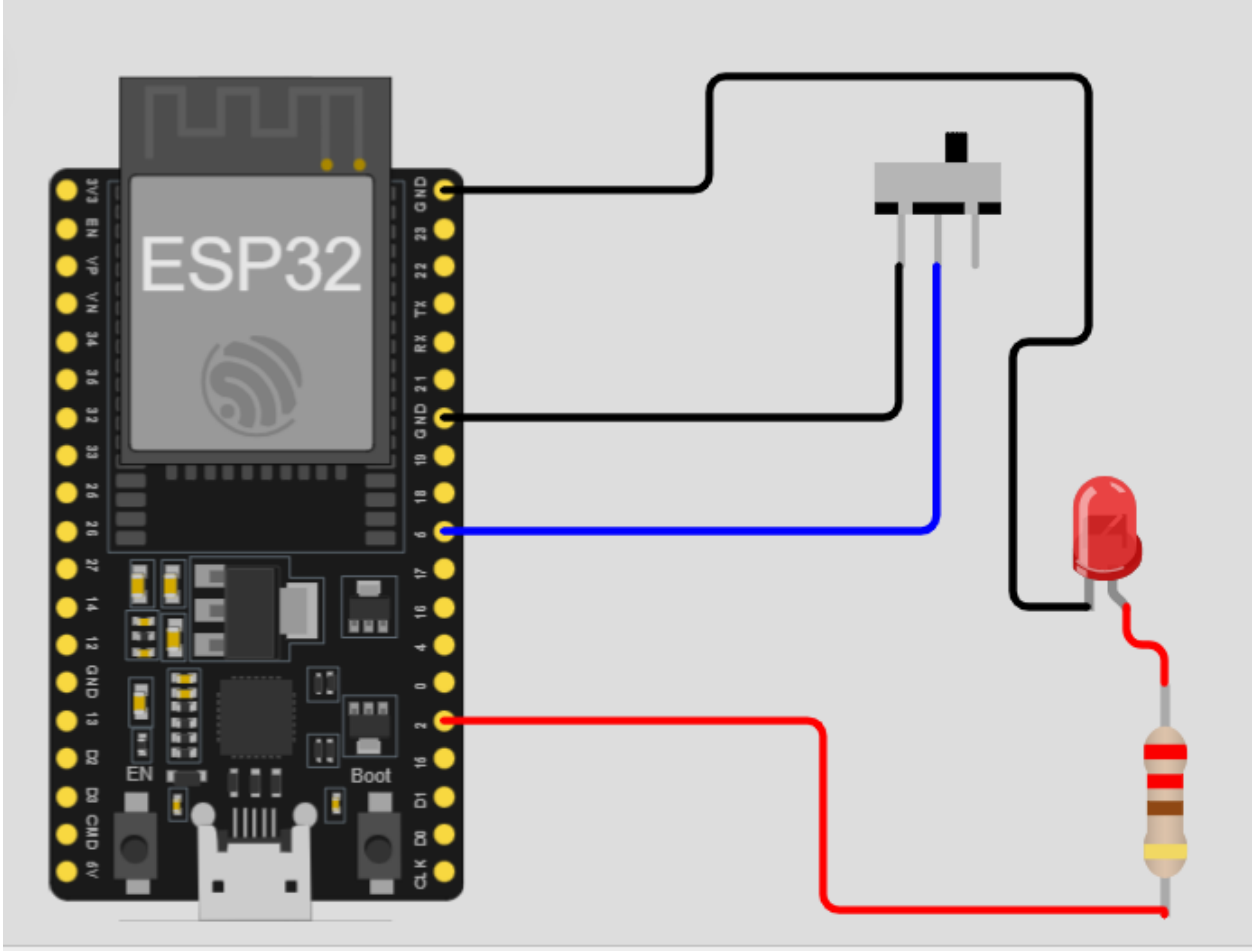
```
const int BUTTON_PIN = 5;  // GIOP5 connected to button
const int LED_PIN    = 2;  // Onboard LED

// variable for storing the pushbutton status
int buttonState = 0;

void setup() {
  Serial.begin(115200);

  // Initialize the pushbutton pin as an input with internal pull-up
  // This means the pin is HIGH by default and LOW when pressed
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  // Initialize the LED pin as an output
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  // Read the state of the pushbutton value
  buttonState = digitalRead(BUTTON_PIN);

  // Check if the pushbutton is pressed.
  // If it is, the buttonState is LOW (because of INPUT_PULLUP)
  if (buttonState == LOW) {
    digitalWrite(LED_PIN, HIGH); // Turn LED on
    Serial.println("Button Pressed!");
  } else {
    digitalWrite(LED_PIN, LOW);  // Turn LED off
  }
}
```



2. Slide Switch / Toggle Switch

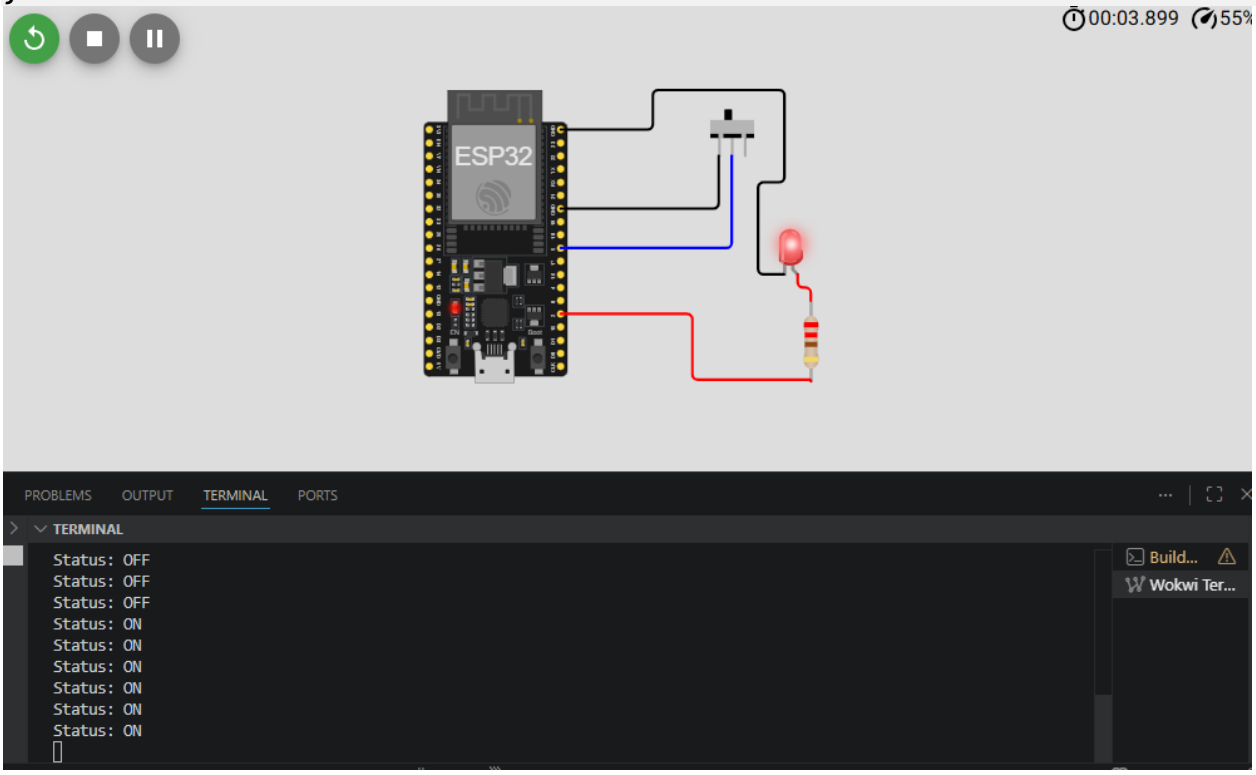
Maintains ON/OFF state for mode selection, power control, or configuration settings.

Pins of device	Pins of ESP32
Middle Pin (2)	GPIO 5
One outer Pin (1)	GND
Other outer pin (3)	-




```
#include <Arduino.h>

// Define the pins
const int SWITCH_PIN = 5;
const int LED_PIN    = 2; // Internal LED

void setup() {
  Serial.begin(115200);

  // Set the switch pin as input with internal pull-up
  pinMode(SWITCH_PIN, INPUT_PULLUP);

  // Set the onboard LED as output
  pinMode(LED_PIN, OUTPUT);

  Serial.println("System Initialized. Flip the switch!");
}

void loop() {
  // Read the current position of the switch
```

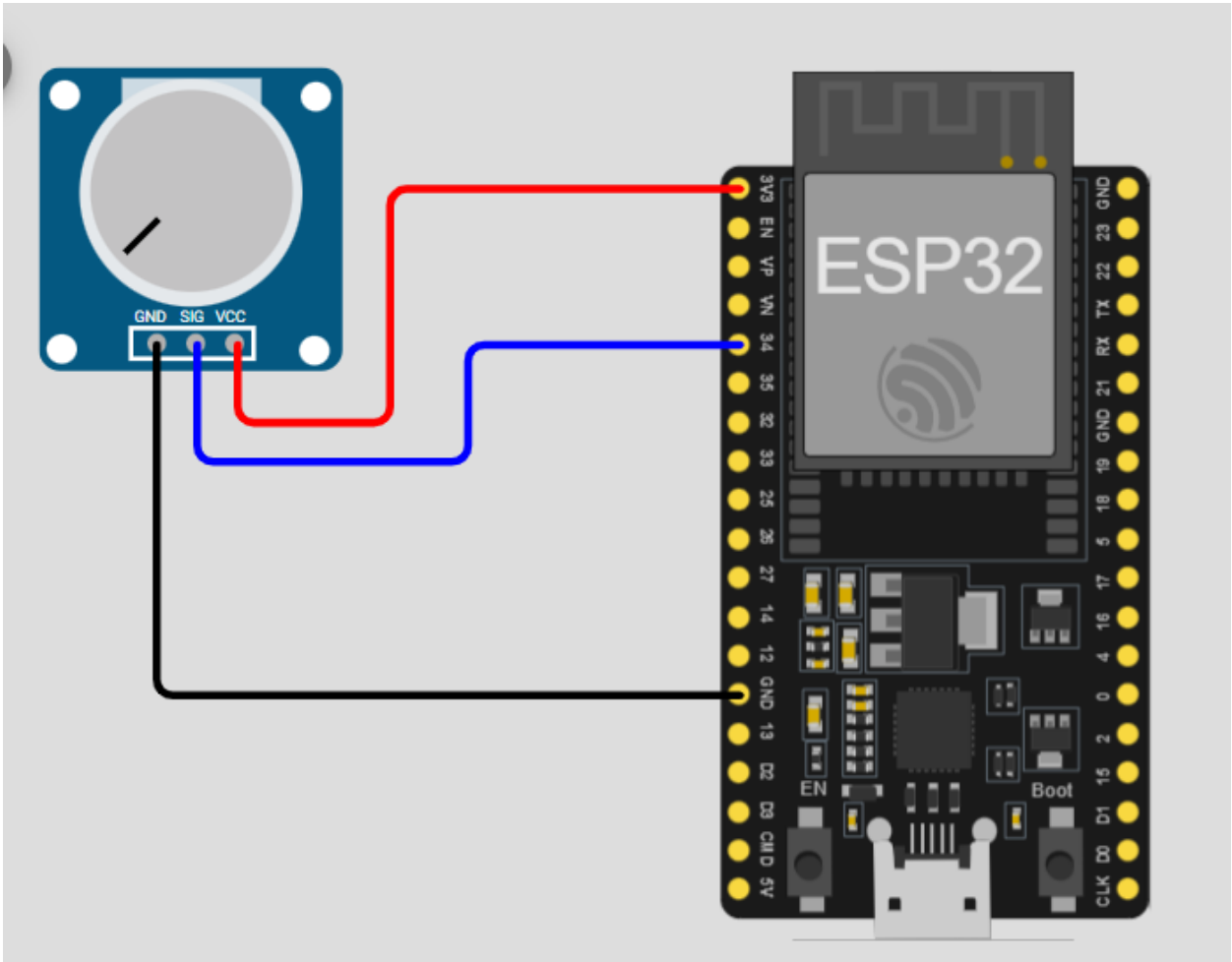
```
  int switchState = digitalRead(SWITCH_PIN);

  if (switchState == LOW) {
    // Switch is closed (connected to GND)
    digitalWrite(LED_PIN, HIGH);
    Serial.println("Status: ON");
  } else {
    // Switch is open (HIGH due to pull-up)
    digitalWrite(LED_PIN, LOW);
    Serial.println("Status: OFF");
  }

  // Small delay to prevent flooding the Serial Monitor
  delay(200);
}
```



3. Potentiometer (Rotary)

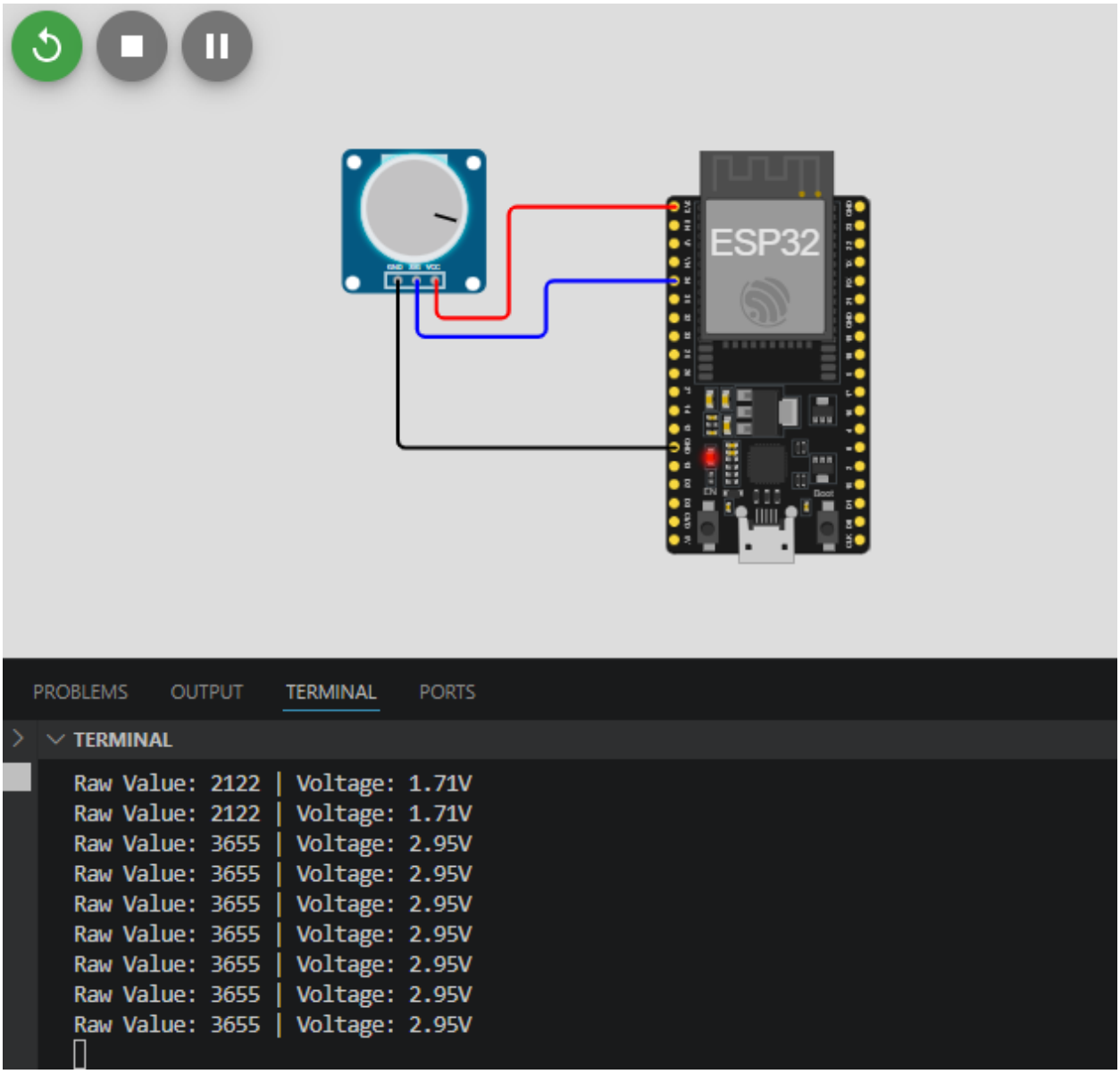
Variable resistor used to adjust analog values (e.g., volume, brightness, speed control).

Pins of device	Pins of ESP32
GND	GND
SIG	GPIO 34
VCC	3.3V



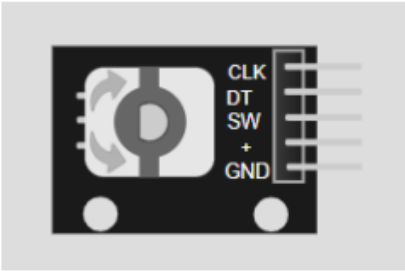

```
#include <Arduino.h>

// ADC1_CH6 is GPIO 34
```

```
const int POT_PIN = 34;

void setup() {
  Serial.begin(115200);

  // Configure ADC resolution (optional, default is 12-bit)
  analogReadResolution(12);

  Serial.println("Potentiometer Read Initialized...");
}

void loop() {
  // Read the analog value (0 - 4095)
  int rawValue = analogRead(POT_PIN);

  // Convert the raw value to a voltage (0 - 3.3V)
  float voltage = (rawValue * 3.3) / 4095.0;

  // Print the results
  Serial.print("Raw Value: ");
  Serial.print(rawValue);
  Serial.print(" | Voltage: ");
  Serial.print(voltage);
  Serial.println("V");

  // Small delay for readability
  delay(250);
}
```

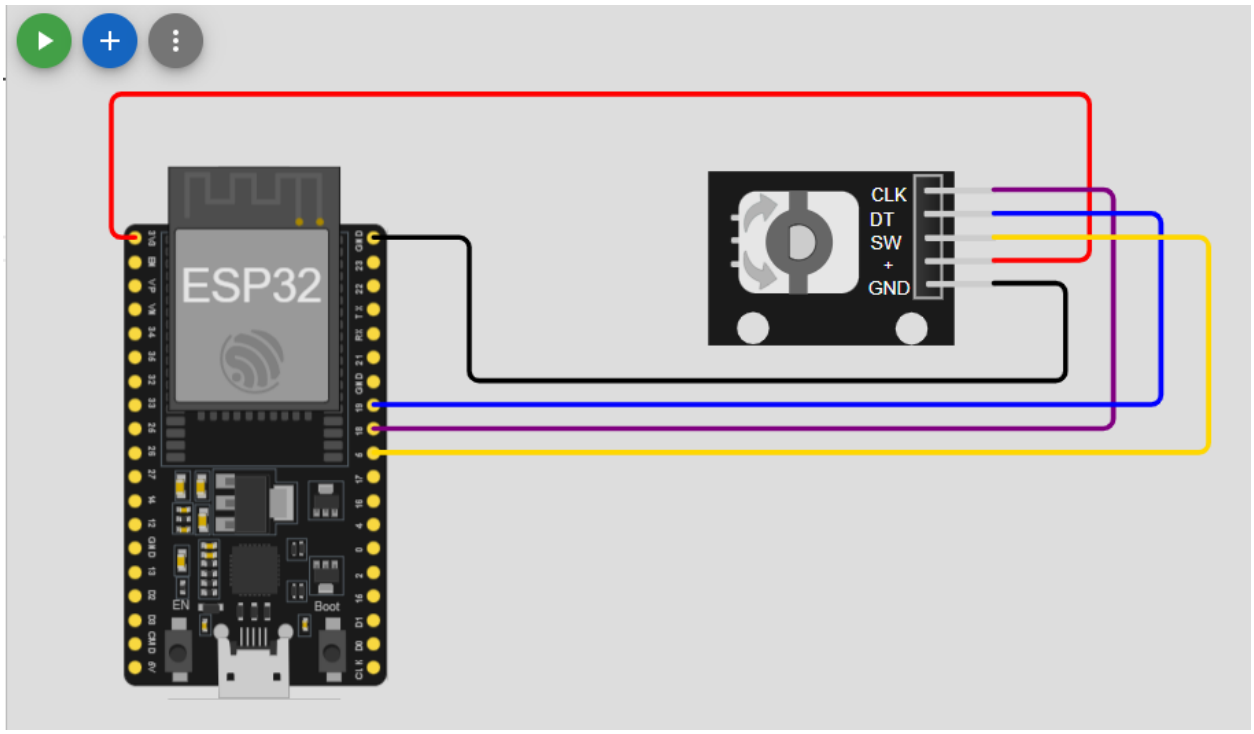


4. Rotary Encoder (KY-040)

Detects rotation direction and steps. Used for menu navigation, volume control, and precise parameter adjustment.

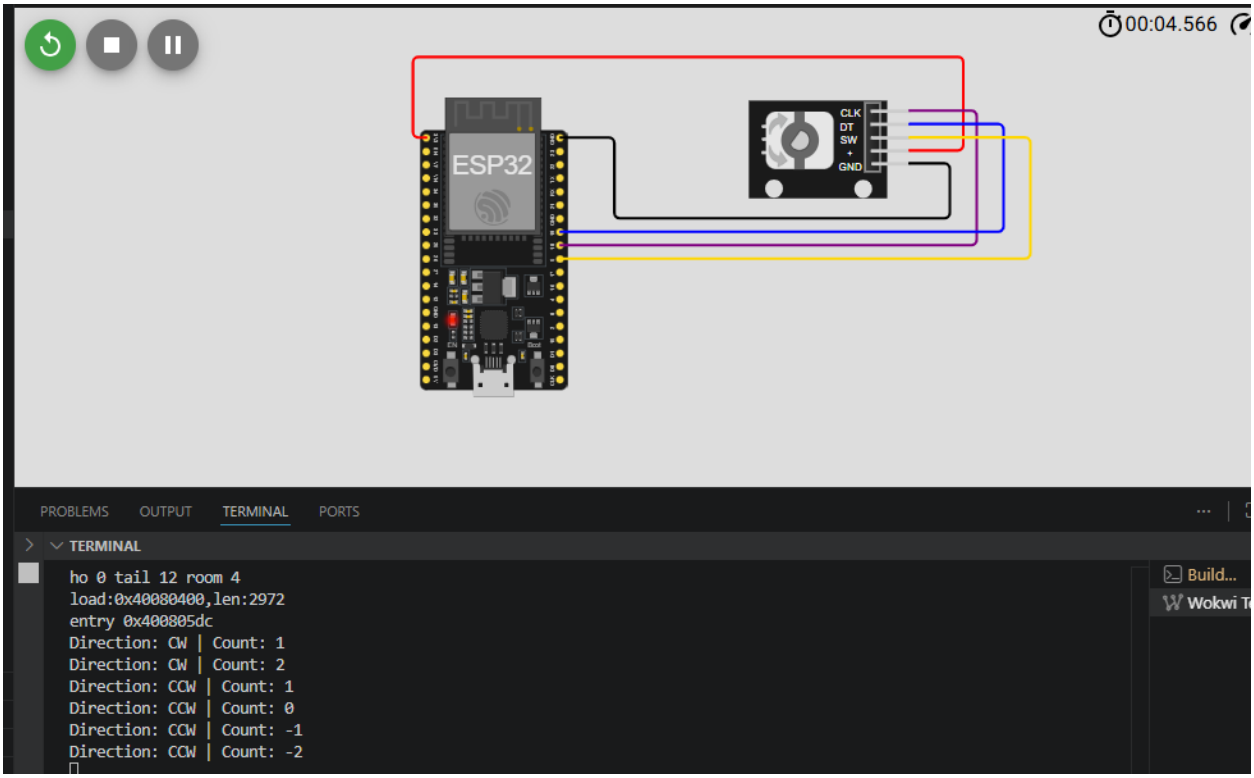


Pins of device	Pins of ESP32
CLK	GPIO 18
DT	GPIO 19
SW	GPIO 5
VCC	3.3V
GND	GND




```
#include <Arduino.h>

// Define pins
#define CLK_PIN 18
#define DT_PIN  19
#define SW_PIN  5

int counter = 0;
int currentStateCLK;
int lastStateCLK;
unsigned long lastButtonPress = 0;

void setup() {
  Serial.begin(115200);

  // Setup pins
  pinMode(CLK_PIN, INPUT);
  pinMode(DT_PIN, INPUT);
  pinMode(SW_PIN, INPUT_PULLUP);

  // Read the initial state of CLK
  lastStateCLK = digitalRead(CLK_PIN);
}

void loop() {
```

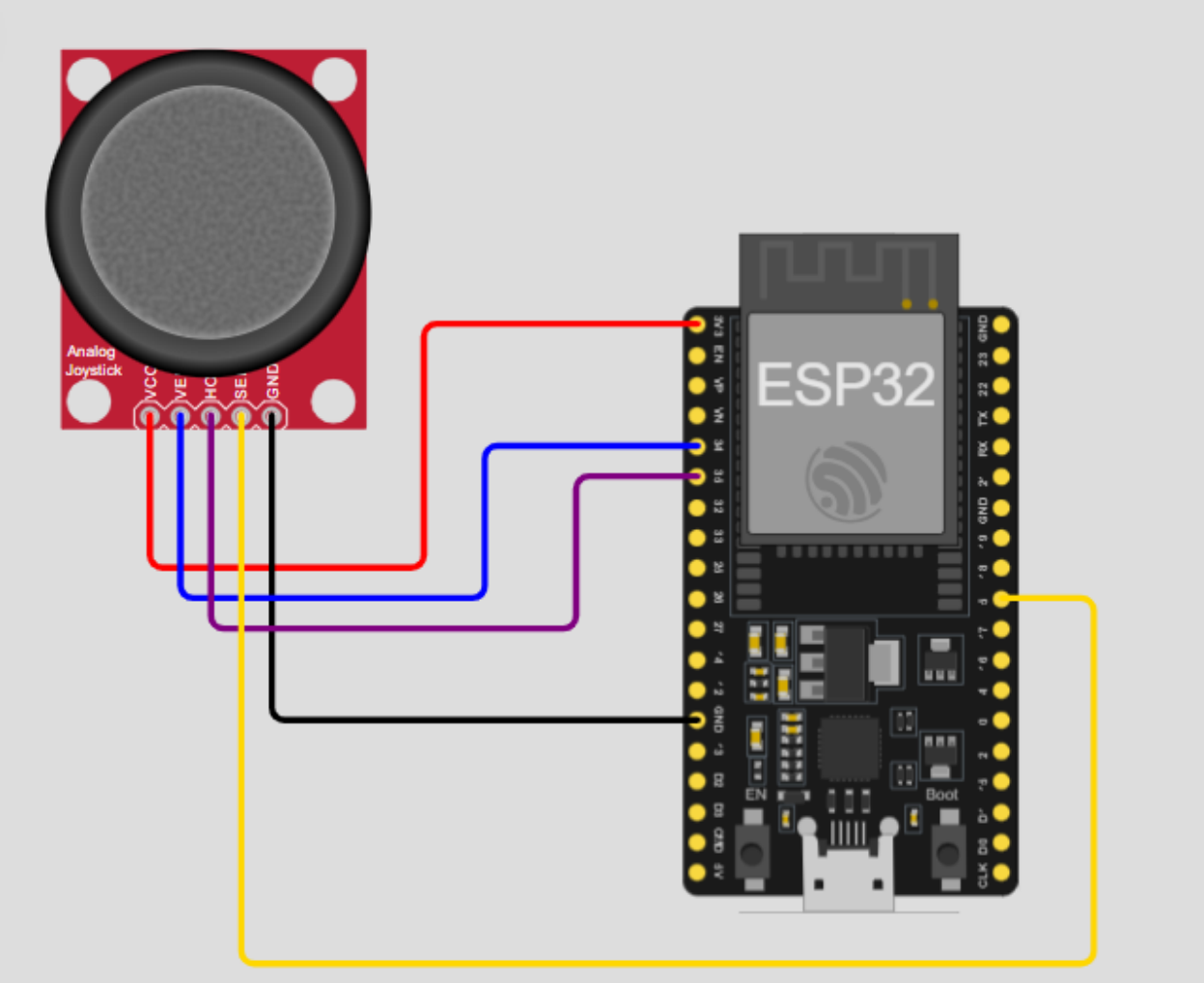
```
  // 1. Read Rotation
  currentStateCLK = digitalRead(CLK_PIN);

  // If CLK has changed, a pulse occurred
  if (currentStateCLK != lastStateCLK && currentStateCLK == 1) {
    // If the DT state is different from the CLK state,
    // the encoder is rotating CCW
    if (digitalRead(DT_PIN) != currentStateCLK) {
      counter--;
    } else {
      // Encoder is rotating CW
      counter++;
    }
    Serial.print("Direction: ");
    Serial.print(digitalRead(DT_PIN) != currentStateCLK ? "CCW" : "CW");
    Serial.print(" | Count: ");
    Serial.println(counter);
  }
  lastStateCLK = currentStateCLK;

  // 2. Read Button Press
  int btnState = digitalRead(SW_PIN);
  if (btnState == LOW) {
    // Basic debouncing
    if (millis() - lastButtonPress > 50) {
      Serial.println("Button Pressed!");
    }
    lastButtonPress = millis();
  }

  // No delay here! We need high-speed polling for the encoder.
}
```



5. Analog Joystick

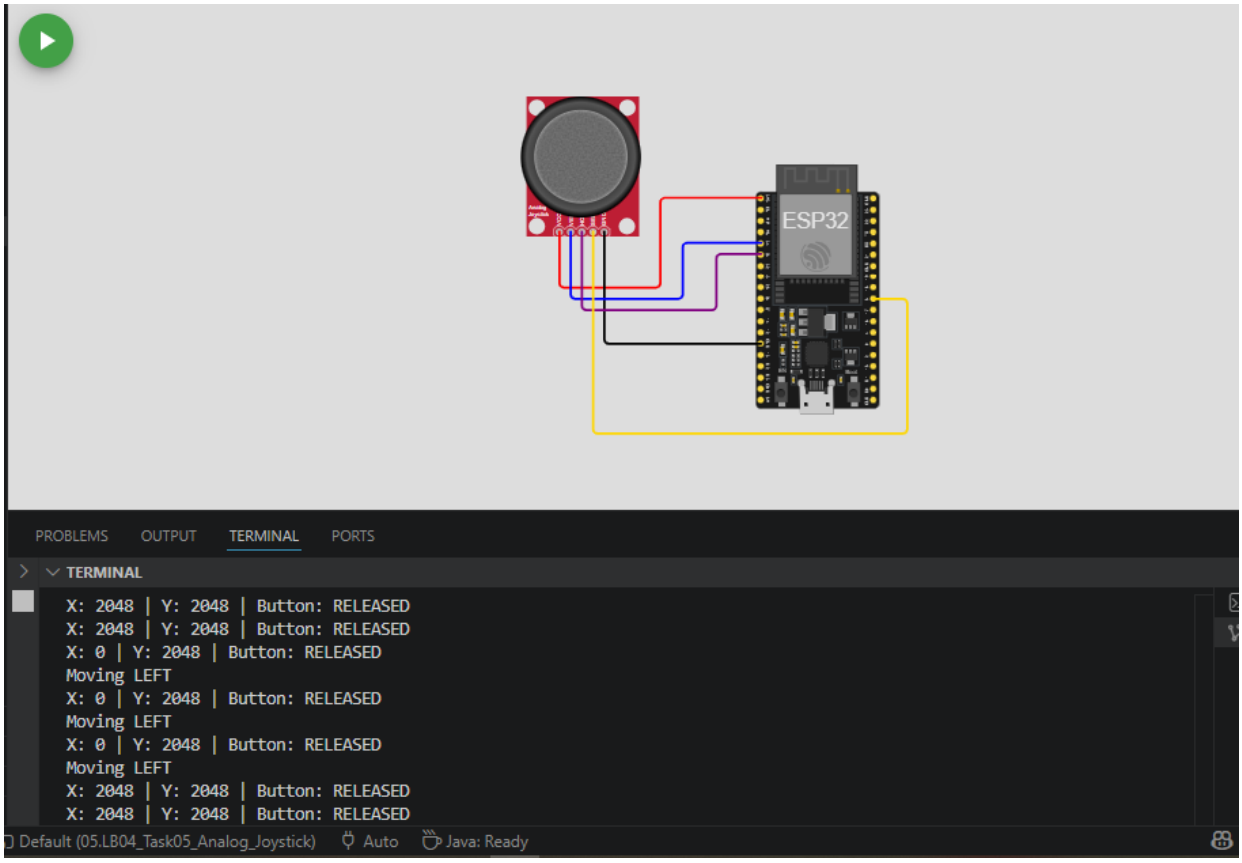
Provides analog X & Y axis movement and a digital button. Used in gaming, robotics control, and pantilt systems.

Pins of device	Pins of ESP32
VCC	3.3V
VERT	GPIO 35
HORZ	GPIO 34
SEL	GPIO 5
GND	GND




```
#include <Arduino.h>

// Define pins
const int PIN_X = 34;
const int PIN_Y = 35;
const int PIN_SW = 5;

void setup() {
  Serial.begin(115200);

  // Joystick button needs a pull-up
  pinMode(PIN_SW, INPUT_PULLUP);

  // Set ADC resolution to 12-bit (0-4095)
  analogReadResolution(12);
}
```

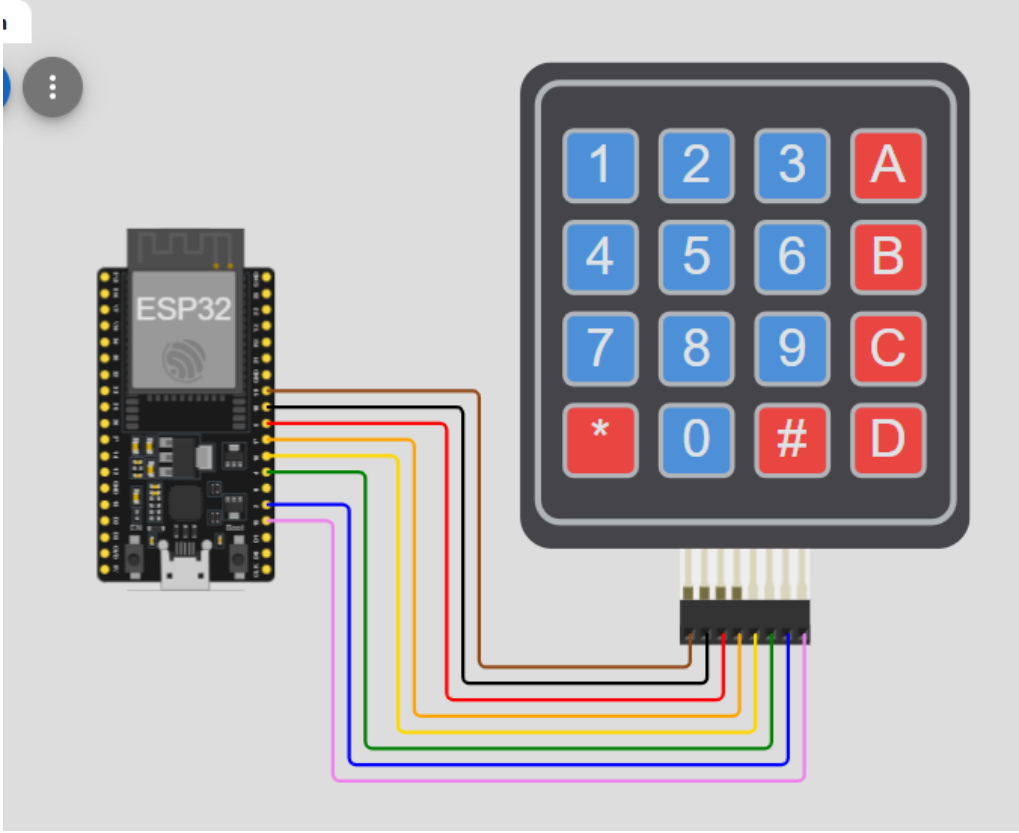
```
void loop() {
  // Read analog values
  int xValue = analogRead(PIN_X);
  int yValue = analogRead(PIN_Y);

  // Read button (inverted logic because of INPUT_PULLUP)
  int swValue = digitalRead(PIN_SW);

  // Print raw values
  Serial.print("X: ");
  Serial.print(xValue);
  Serial.print(" | Y: ");
  Serial.print(yValue);
  Serial.print(" | Button: ");
  Serial.println(swValue == LOW ? "PRESSED" : "RELEASED");

  // Logic Example: Simple Direction detection
  if (xValue < 1500) Serial.println("Moving LEFT");
  else if (xValue > 2500) Serial.println("Moving RIGHT");

  if (yValue < 1500) Serial.println("Moving UP");
  else if (yValue > 2500) Serial.println("Moving DOWN");

  delay(200);
}
```

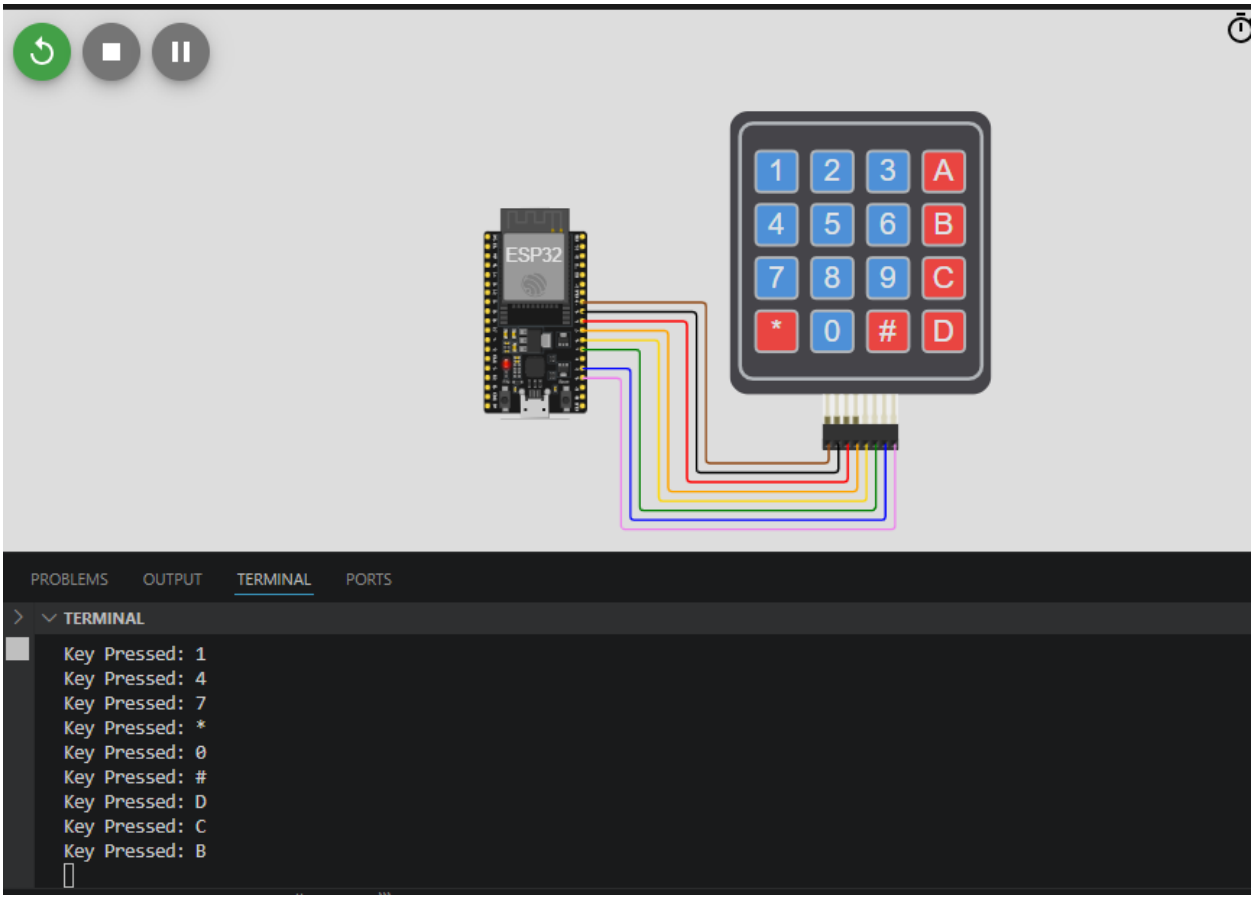


6.Keypad

Allows numeric and character input. Used in password systems, calculators, and menu-based projects.

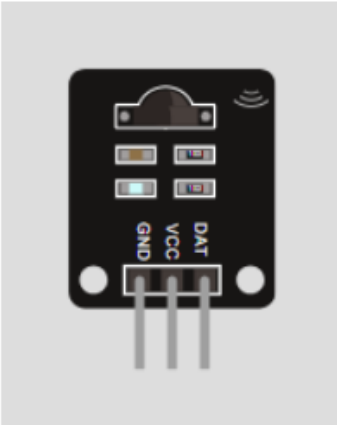
Pins of device	Pins of ESP32
R1	GPIO 19
R2	GPIO 18
R3	GPIO 5
R4	GPIO 17
C1	GPIO 16
C2	GPIO 4
C3	GPIO 2
C4	GPIO 15



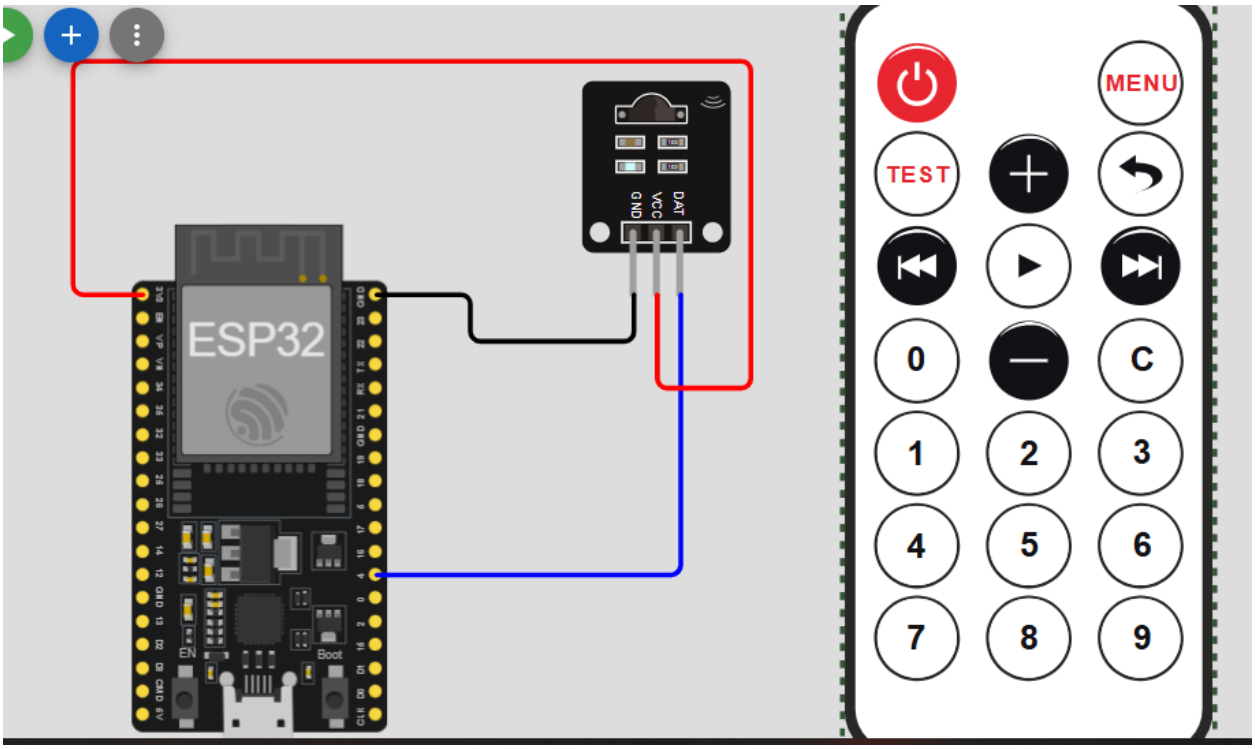


7. IR Receiver

Detects and receives infrared signals from an IR Remote. Used for wireless remote control of devices (TVs, robots, home automation, etc.).

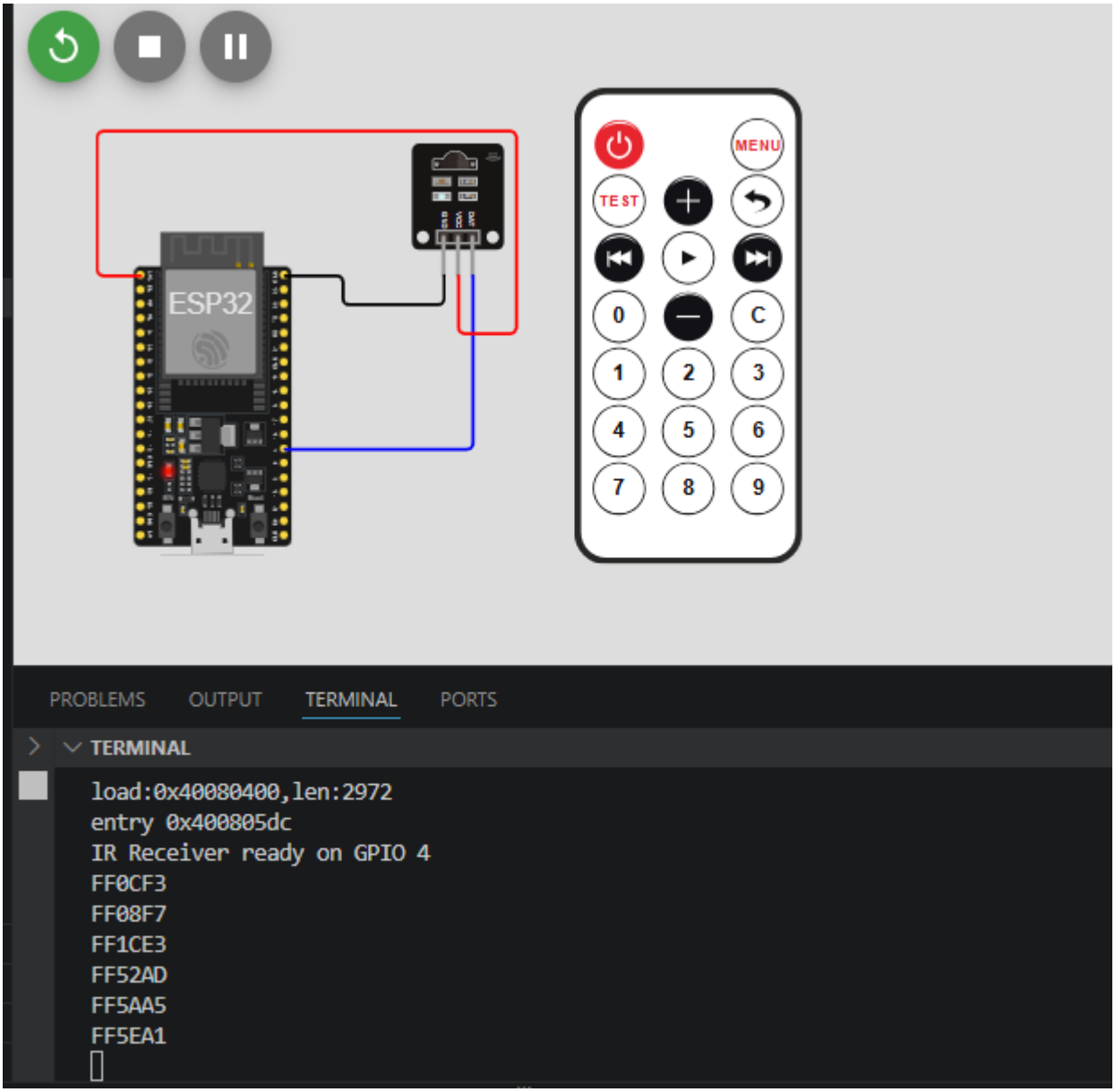


Pins of device	Pins of ESP32
GND	GND
VCC	3.3V
DAT	GPIO 4




```
#include <Arduino.h>
#include <IRremoteESP8266.h>
#include <IRrecv.h>
#include <IRutils.h>

// 1. Define the GPIO pin connected to the IR sensor
const uint16_t kRecvPin = 4;

// 2. Establish the IR receiver object
IRrecv irrecv(kRecvPin);
decode_results results;

void setup() {
  Serial.begin(115200);

  // 3. Start the receiver
  irrecv.enableIRIn();
  Serial.print("IR Receiver ready on GPIO ");
  Serial.println(kRecvPin);
}

void loop() {
  // 4. Check if a signal was received
  if (irrecv.decode(&results)) {
    // Print the protocol (NEC, Sony, etc.) and the Hex code
```

```
    serialPrintUint64(results.value, HEX);
    Serial.println("");

    // Resume listening for the next signal
    irecv.resume();
  }
  delay(100);
}
```



Output Devices

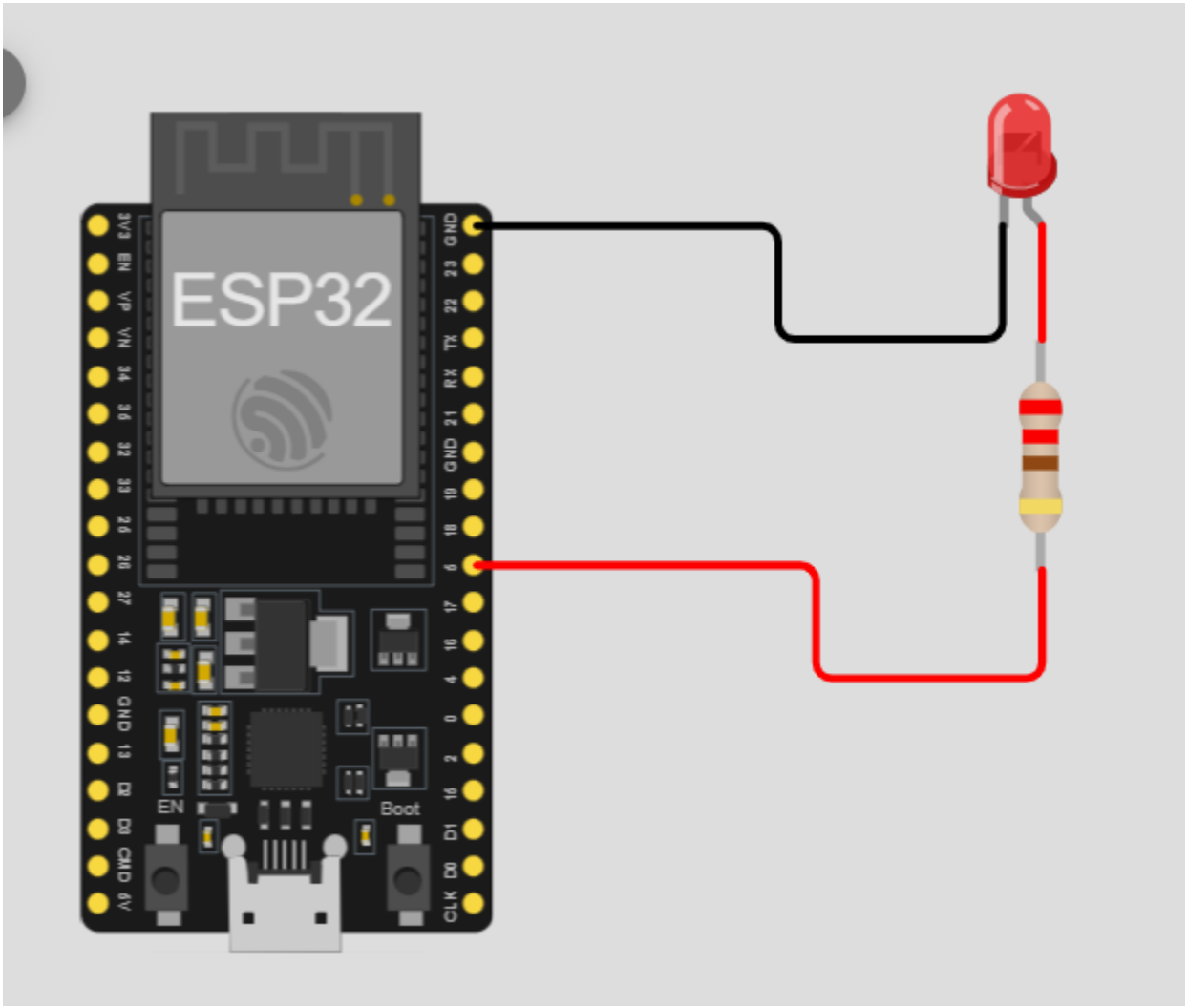
1. IR Remote

It acts like a TV remote. It transmits (sends) invisible infrared light signals when you press its buttons.

2. LED

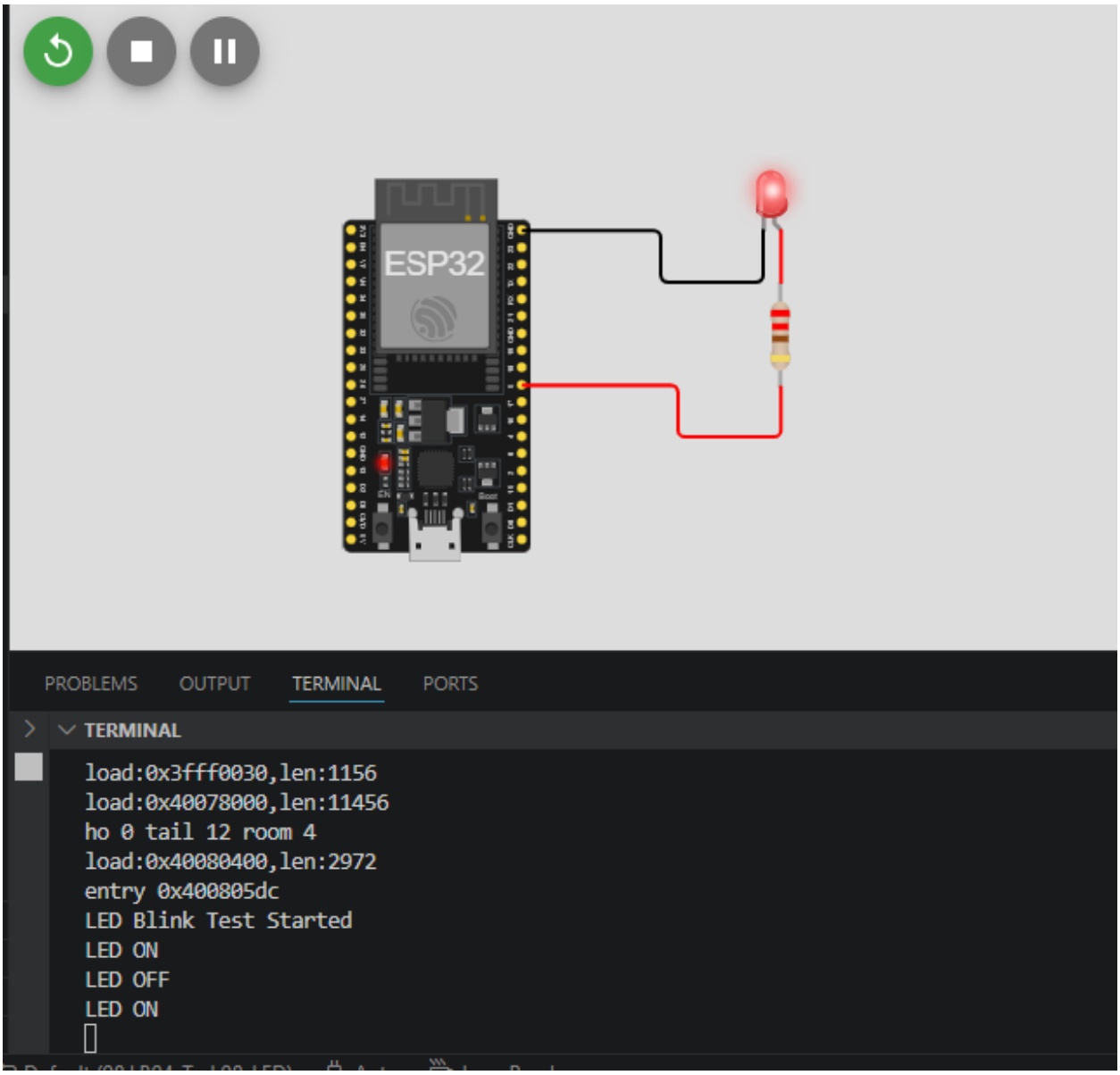
Visual indicator for status, alerts, or debugging.

Pins of device	Pins of ESP32
C	GND
A	GPIO 5 (via 220Ω)




```
#include <Arduino.h>
// Define the LED pin
const int ledPin = 5;

void setup() {
  // Initialize the digital pin as an output
  pinMode(ledPin, OUTPUT);
  Serial.begin(115200);
  Serial.println("LED Blink Test Started");
}

void loop() {
  // Turn the LED on (HIGH is the voltage level)
  digitalWrite(ledPin, HIGH);
  Serial.println("LED ON");
  delay(1000); // Wait for a second
```

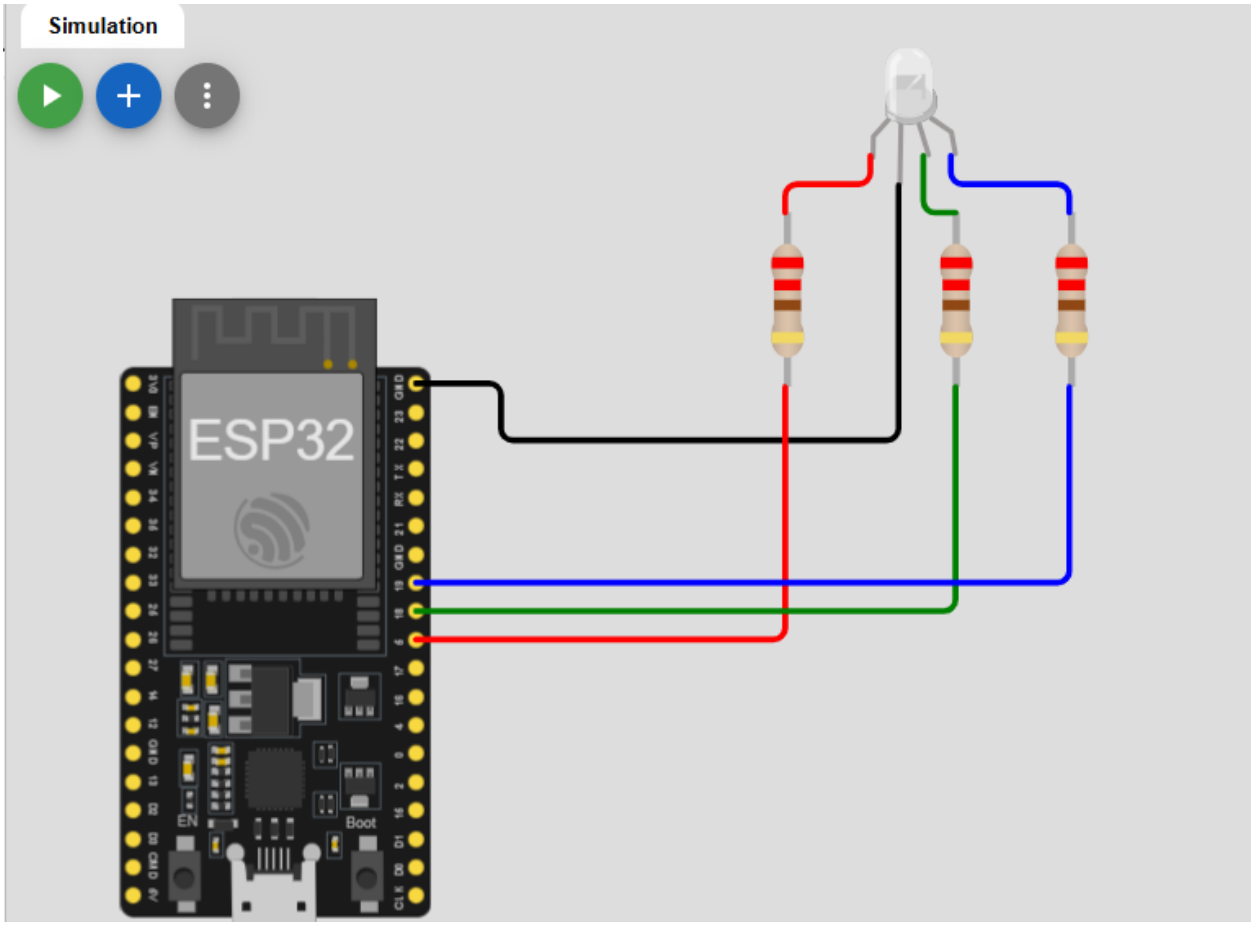
```
  // Turn the LED off by making the voltage LOW
  digitalWrite(ledPin, LOW);
  Serial.println("LED OFF");
  delay(1000); // Wait for a second
}
```



3. RGB LED

Produces multiple colors for status indication or decorative lighting.

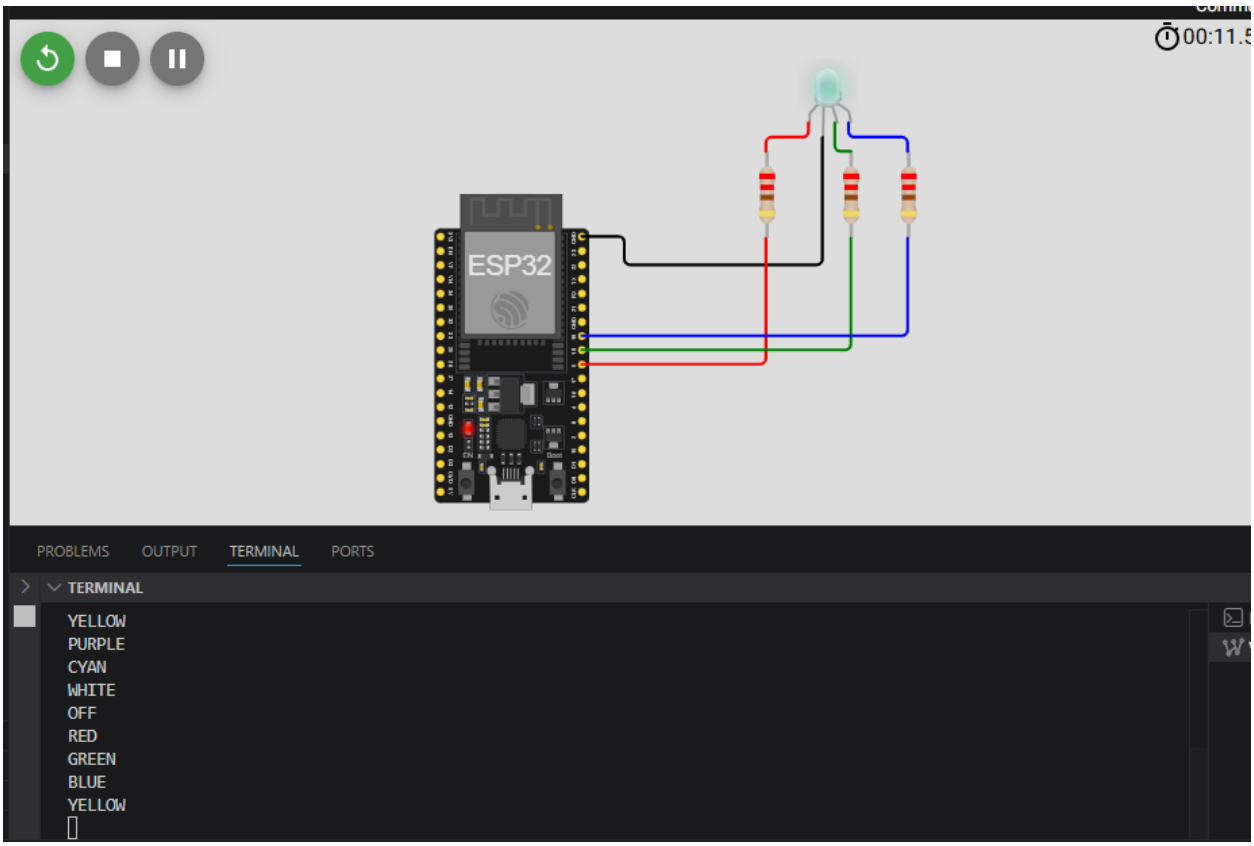
Pins of device	Pins of ESP32
R	GPIO 5 (via 220Ω)
COM	GND
G	GPIO 18 (via 220Ω)
B	GPIO 19 (via 220Ω)




```
#include <Arduino.h>

// Define Pins
const int redPin = 5;
```

```
const int greenPin = 18;
const int bluePin = 19;

// PWM Settings
const int freq = 5000;
const int resolution = 8; // 8-bit resolution (0-255)

// PWM Channels
const int redChannel = 0;
const int greenChannel = 1;
const int blueChannel = 2;
const bool commonAnode = false;

// Forward declaration
void setColor(int redValue, int greenValue, int blueValue);

void setup() {
  Serial.begin(115200);

  // Set pins as outputs
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);

  // Configure LED PWM functionalites
  ledcSetup(redChannel, freq, resolution);
  ledcSetup(greenChannel, freq, resolution);
  ledcSetup(blueChannel, freq, resolution);

  ledcAttachPin(redPin, redChannel);
  ledcAttachPin(greenPin, greenChannel);
  ledcAttachPin(bluePin, blueChannel);

  // Startup check: LED should turn white briefly.
  setColor(255, 255, 255);
  delay(1000);
  setColor(0, 0, 0);

  Serial.println("RGB LED initialized!");
}

void loop() {
  // Cycle through some colors
  Serial.println("RED");
  setColor(255, 0, 0);   // Red
```

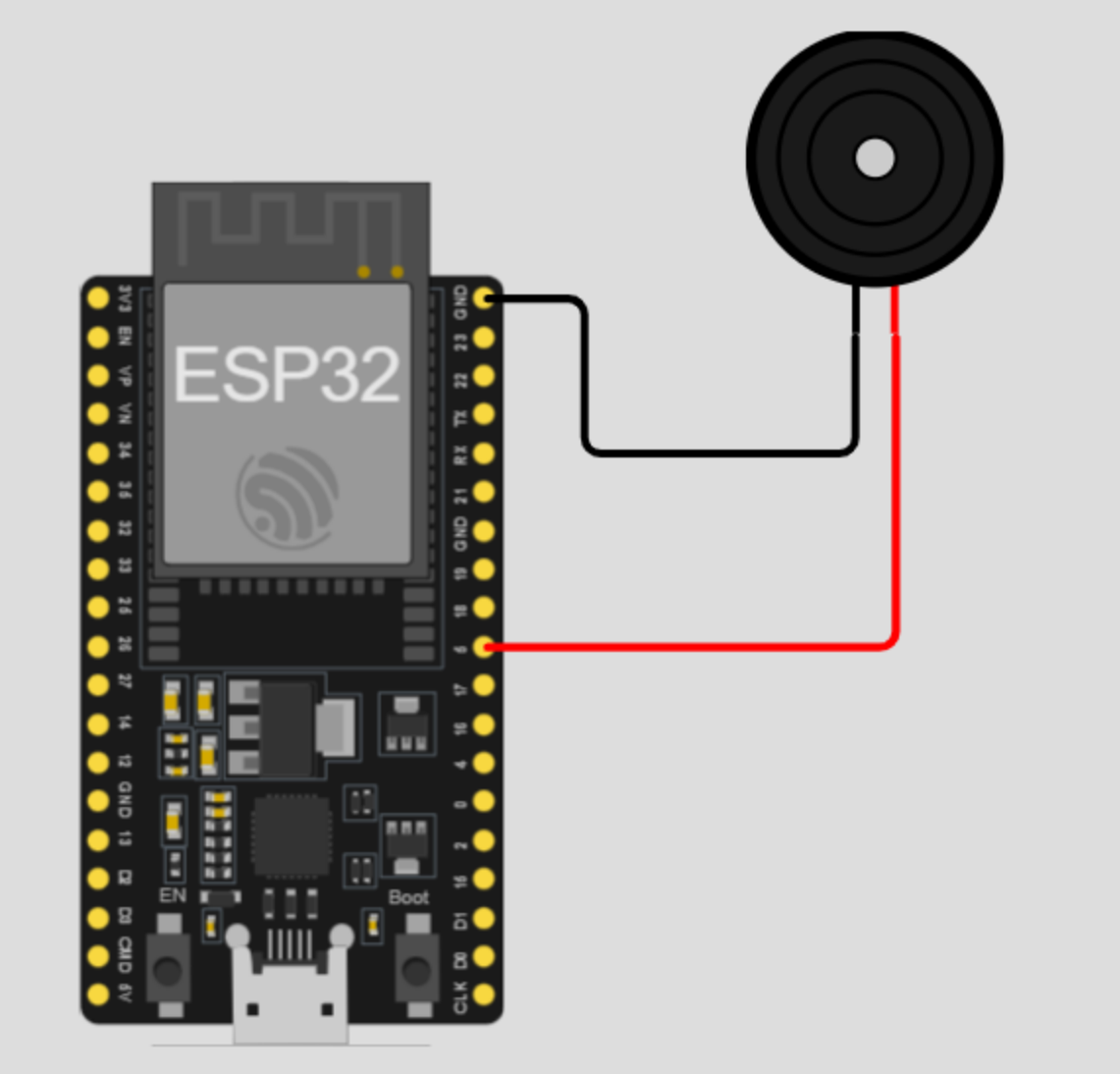
```
  delay(1000);

  Serial.println("GREEN");
  setColor(0, 255, 0);   // Green
  delay(1000);

  Serial.println("BLUE");
  setColor(0, 0, 255);   // Blue
  delay(1000);

  Serial.println("YELLOW");
  setColor(255, 255, 0); // Yellow
  delay(1000);

  Serial.println("PURPLE");
  setColor(255, 0, 255); // Purple
  delay(1000);

  Serial.println("CYAN");
  setColor(0, 255, 255); // Cyan
  delay(1000);

  Serial.println("WHITE");
  setColor(255, 255, 255); // White
  delay(1000);

  Serial.println("OFF");
  setColor(0, 0, 0);     // Off
  delay(1000);
}

// Function to set the RGB color
void setColor(int redValue, int greenValue, int blueValue) {
  if (commonAnode) {
    redValue = 255 - redValue;
    greenValue = 255 - greenValue;
    blueValue = 255 - blueValue;
  }

  ledcWrite(redChannel, redValue);
  ledcWrite(greenChannel, greenValue);
  ledcWrite(blueChannel, blueValue);
}
```



4. Buzzer

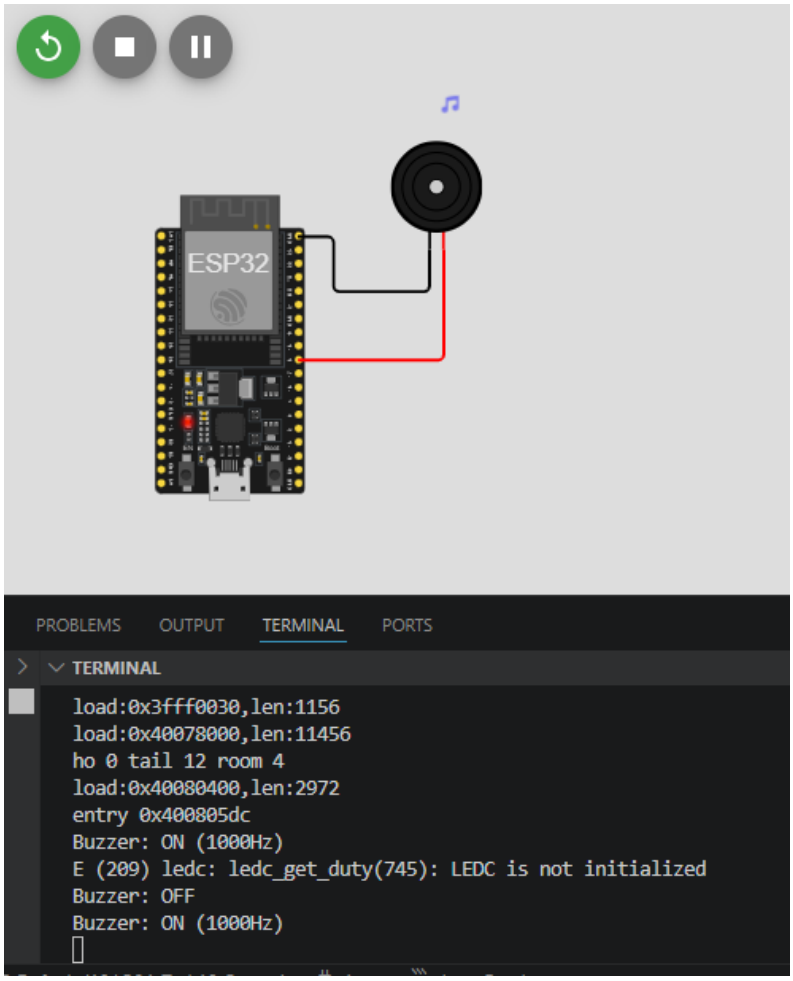
Produces simple beeps or tones for alerts, notifications, or feedback.

Pins of device	Pins of ESP32
1	GND
2	GPIO 5



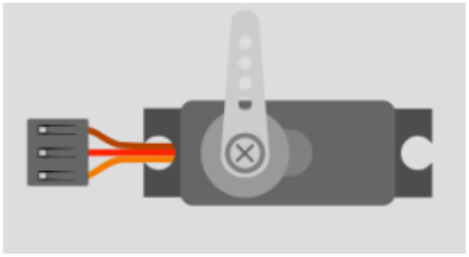

```
#include <Arduino.h>
// Define the buzzer pin
const int buzzerPin = 5;

void playMelody() {
  int notes[] = {262, 294, 330, 349}; // C4, D4, E4, F4
  for (int i = 0; i < 4; i++) {
    tone(buzzerPin, notes[i]);
    delay(200);
    noTone(buzzerPin);
    delay(50);
  }
}

void setup() {
```

```
  // No special setup needed for tone()
  Serial.begin(115200);
}

void loop() {
  Serial.println("Buzzer: ON (1000Hz)");

  // Play a 1000Hz tone
  tone(buzzerPin, 1000);
  delay(500);

  Serial.println("Buzzer: OFF");

  // Stop the tone
  noTone(buzzerPin);
  delay(1000);

  // Simple melody test (Passive Buzzer only)
  playMelody();
}
```

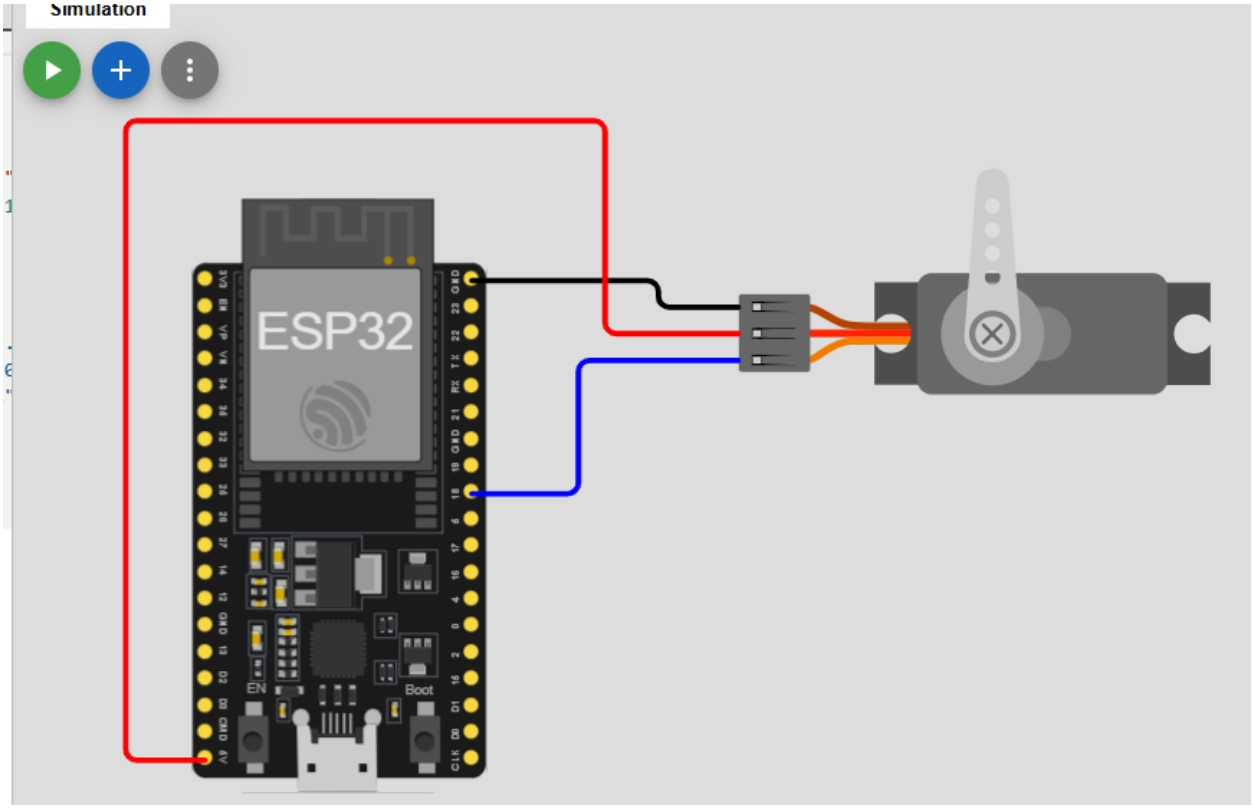


5. Servo Motor

Precise angular control (0–180°). Used in robotics, camera gimbals, and door locks.

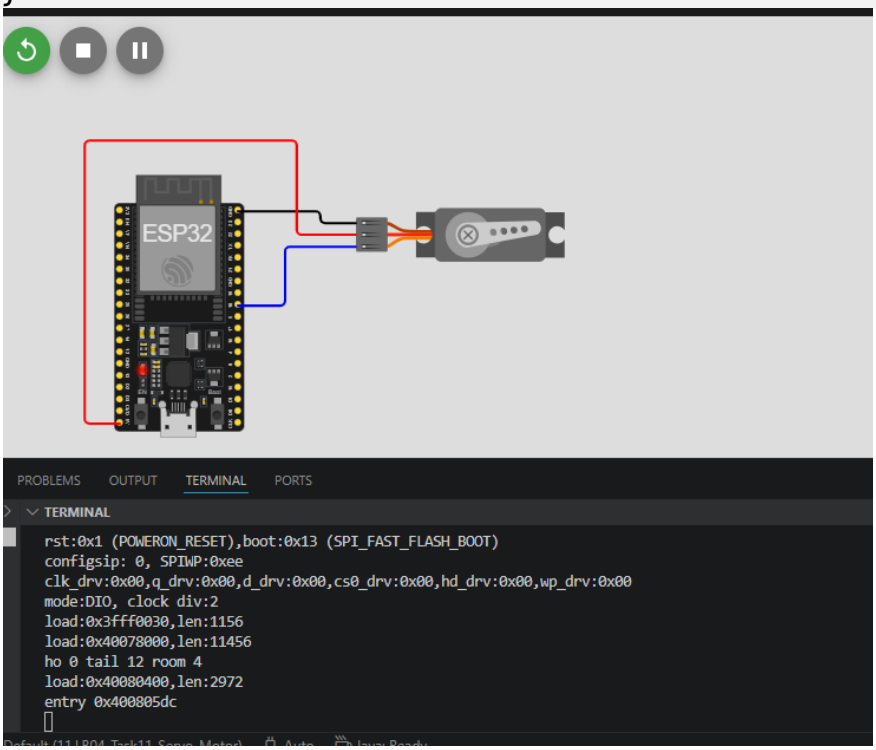


Pins of device	Pins of ESP32
GND	GND
V+	5V
PWM	GPIO 18



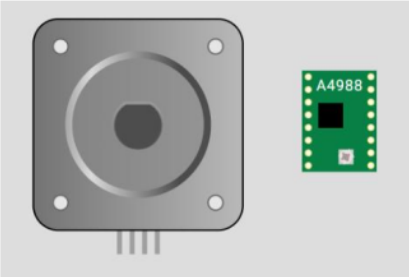

```
#include <Arduino.h>
#include <ESP32Servo.h>

// 1. Create a servo object
Servo myServo;

// 2. Define the GPIO pin
const int servoPin = 18;
```

```
void setup() {
  Serial.begin(115200);

  // 3. Attach the servo to the pin
  // Standard servos use a pulse width of 500us to 2400us
  myServo.attach(servoPin, 500, 2400);
}

void loop() {
  // Move from 0 to 180 degrees
  for (int pos = 0; pos <= 180; pos += 1) {
    myServo.write(pos);
    delay(15); // Wait for the servo to reach the position
  }

  delay(1000);

  // Move from 180 to 0 degrees
  for (int pos = 180; pos >= 0; pos -= 1) {
    myServo.write(pos);
    delay(15);
  }

  delay(1000);
}
```

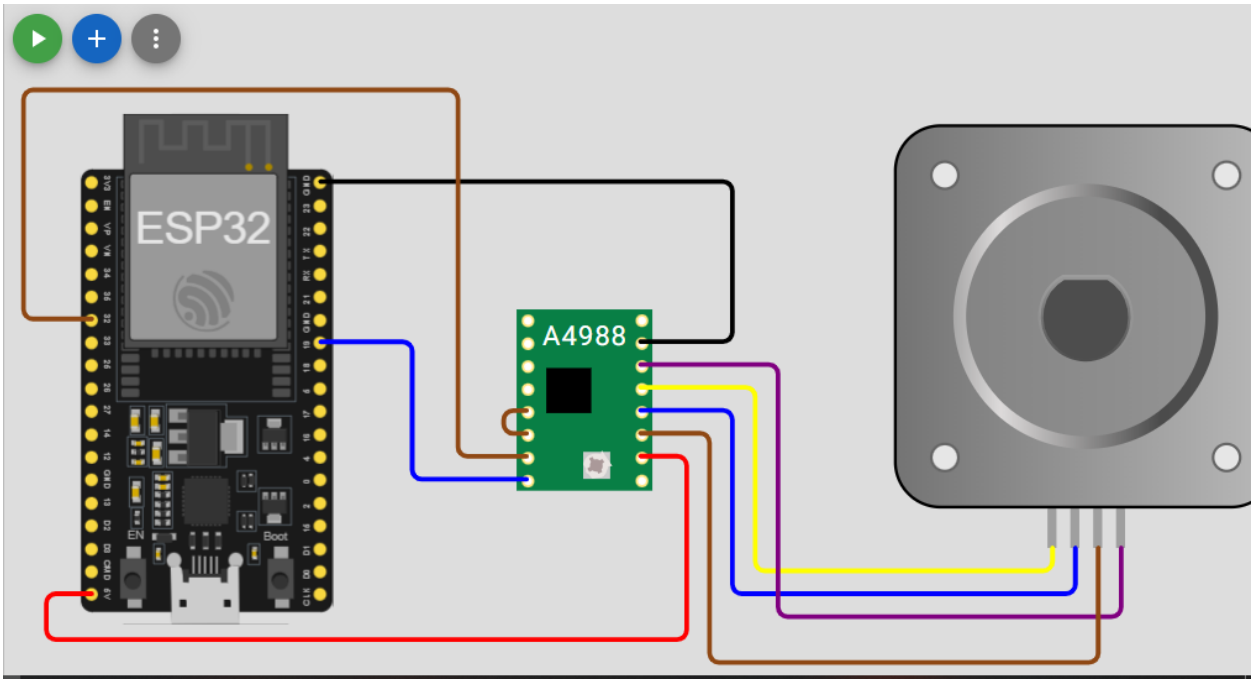


6. Bipolar Stepper Motor + A4988 Driver

Provides precise angular movement and position control. Used in robotics, 3D printers, CNC machines, and automated positioning systems.

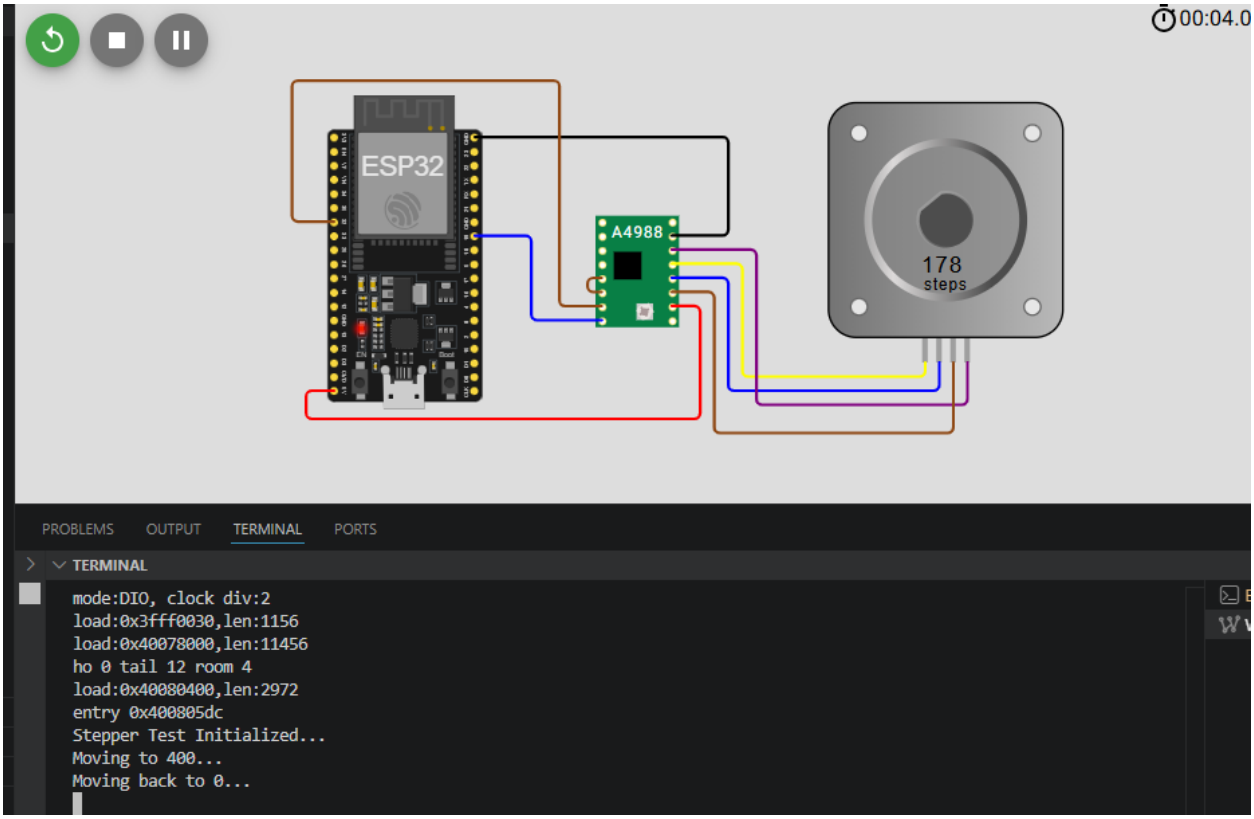


Pins of Stepper motor	Pins of A4988 driver	Pins of ESP32
A-	2A	
A+	1A	
B+	1B	
B-	2B	
	RESET	Connect Together
	SLEEP	
	STEP	GPIO 32
	DIR	GPIO 19
	VDD	5V
	GND	GND




```
#include <Arduino.h>
#include <AccelStepper.h>
```

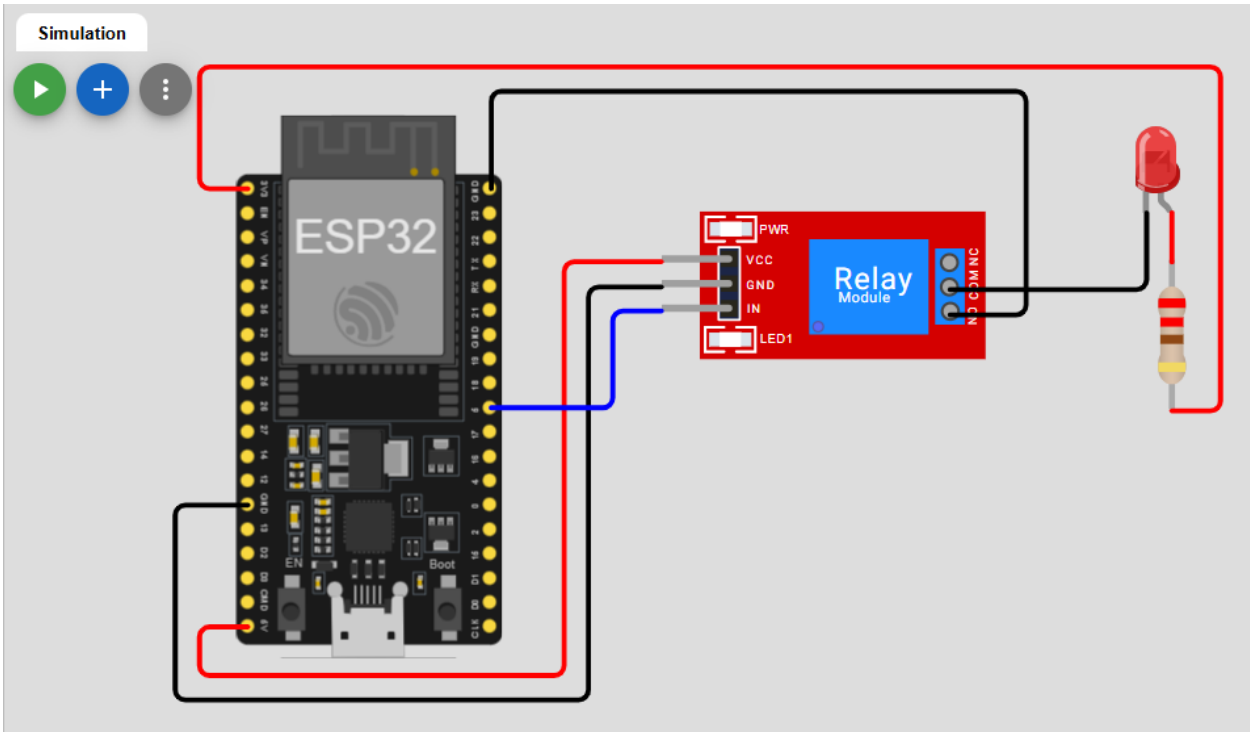
```
// 1. Define the pins
const int stepPin = 32;
const int dirPin = 19;

// 2. Define the motor interface type (1 means a driver like A4988)
#define motorInterfaceType 1

// 3. Create the stepper object
AccelStepper stepper(motorInterfaceType, stepPin, dirPin);

void setup() {
  Serial.begin(115200);

  // 4. Set maximum speed and acceleration
  stepper.setMaxSpeed(1000);     // Steps per second
  stepper.setAcceleration(500);  // Steps per second squared

  Serial.println("Stepper Test Initialized...");
}

void loop() {
  // Move to a position (e.g., 200 steps is usually one full rotation)
  Serial.println("Moving to 400...");
  stepper.moveTo(400);
  stepper.runToPosition(); // Blocking call: waits until finished

  delay(1000);

  // Move back to start
  Serial.println("Moving back to 0...");
  stepper.moveTo(0);
  stepper.runToPosition();

  delay(1000);
}
```



7. Relay Module

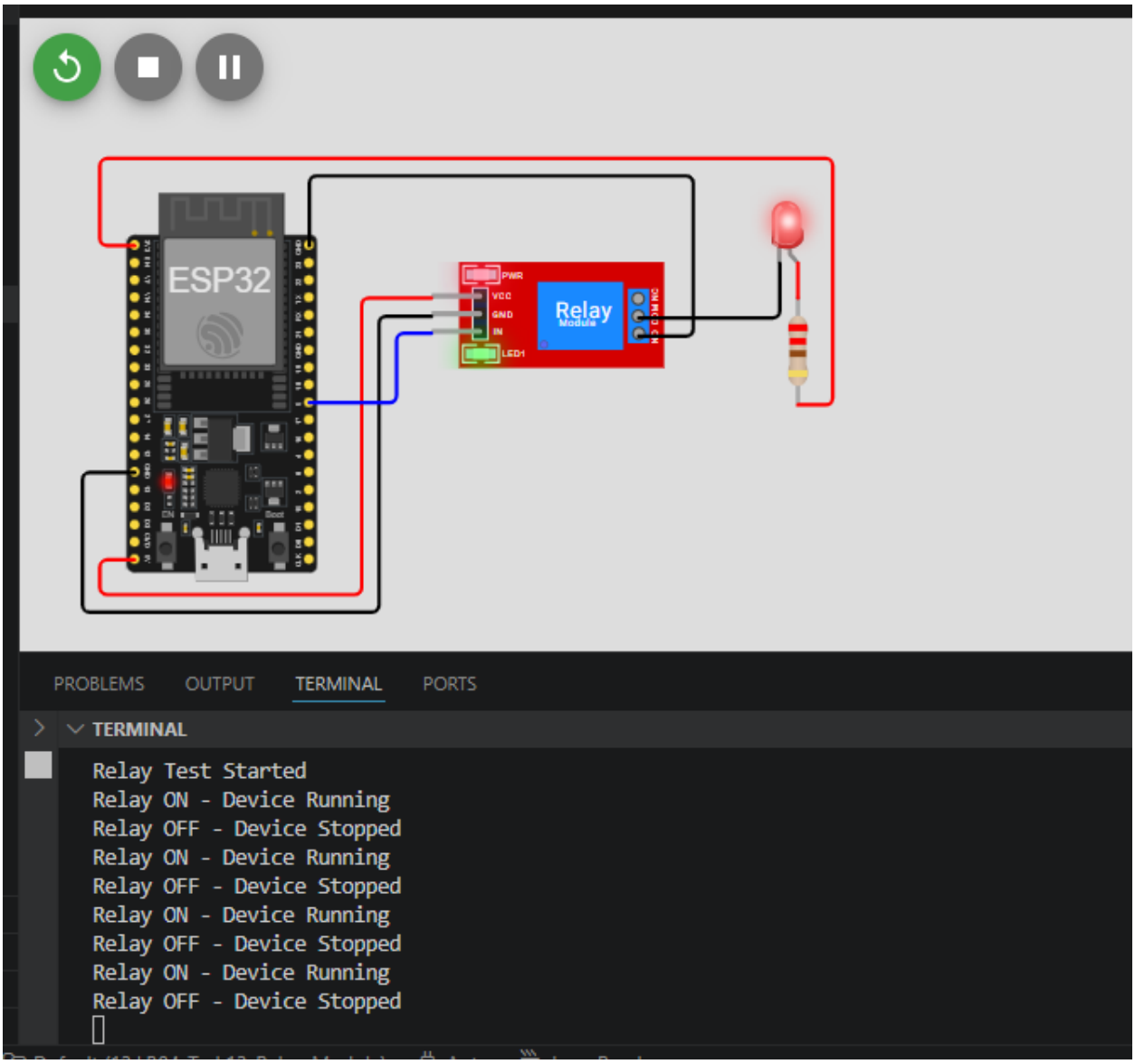
Allows low-power ESP32 to control high-power devices (lights, motors, appliances).

	Pins of device	Pins of ESP32
Input	VCC	5V
	GND	GND
	IN	GPIO 5
Output	NC	-
	COM	GND PIN of Output device
	NO	GND



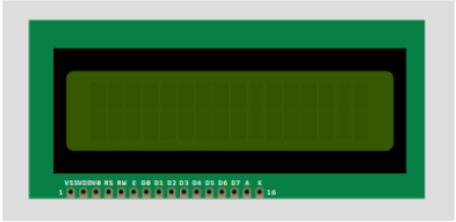

```
#include <Arduino.h>

// Define the Relay Pin
const int relayPin = 5;

void setup() {
  // Initialize the relay pin as an output
  pinMode(relayPin, OUTPUT);

  // Set initial state to OFF (for Active Low relays)
  digitalWrite(relayPin, HIGH);

  Serial.begin(115200);
  Serial.println("Relay Test Started");
}

void loop() {
  // Turn the relay ON
  Serial.println("Relay ON - Device Running");
  digitalWrite(relayPin, LOW);
  delay(3000); // Keep it on for 3 seconds

  // Turn the relay OFF
  Serial.println("Relay OFF - Device Stopped");
  digitalWrite(relayPin, HIGH);
  delay(3000); // Keep it off for 3 seconds
```

```
}
```

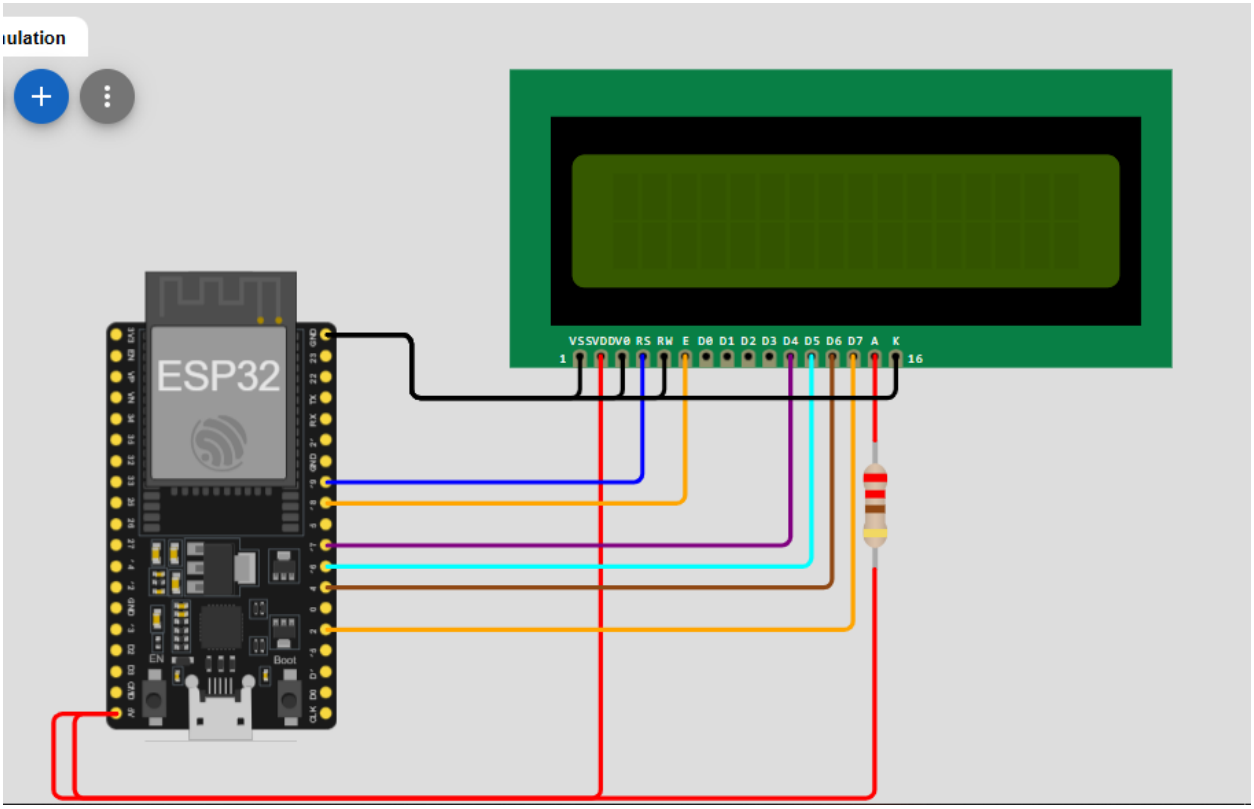


8. LCD 16x2 (Parallel)

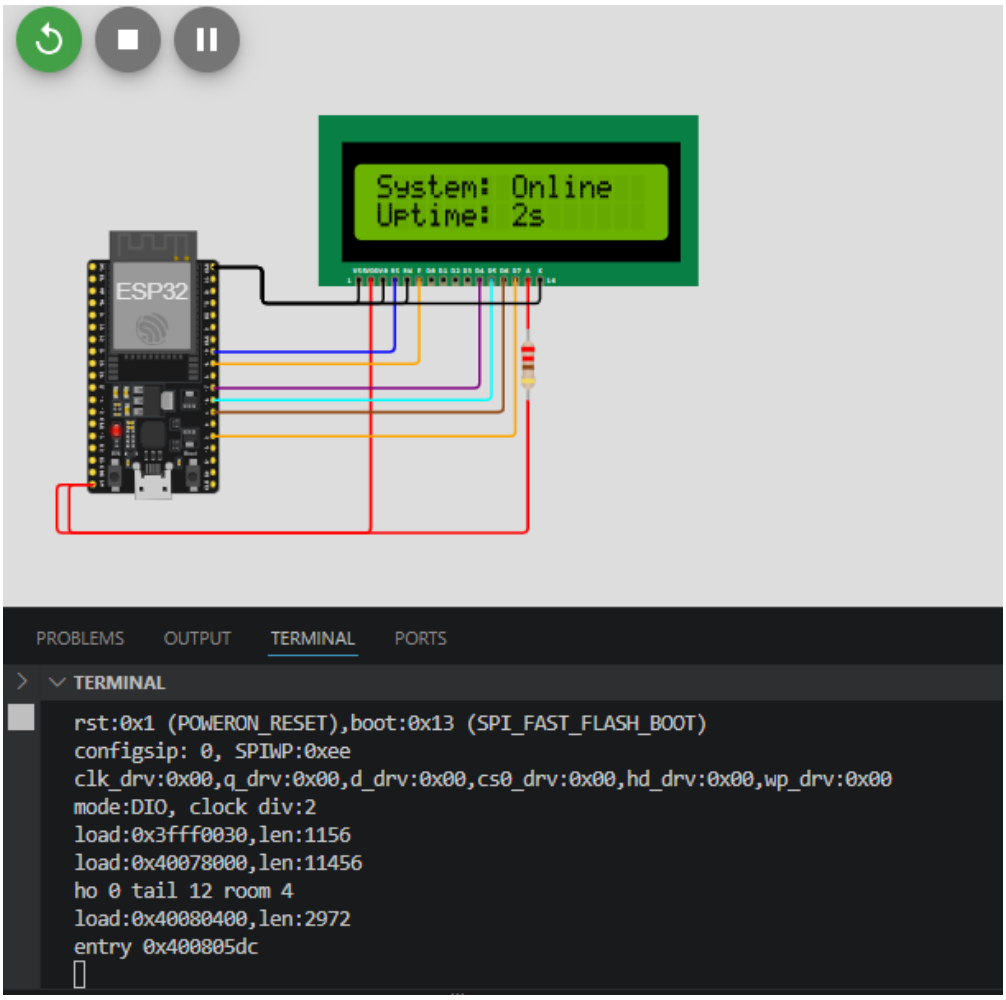
Displays text, numbers, and simple messages (2 rows × 16 characters). Classic and very common in beginner projects.



Pins of device	Pins of ESP32
VSS	GND
VDD	5V
V0	GND (via resistor)
RS	GPIO 19
RW	GND
E	GPIO 18
D0	-
D1	-
D2	-
D3	-
D4	GPIO 17
D5	GPIO 16
D6	GPIO 4
D7	GPIO 2
A	5V (via 220Ω)
K	GND

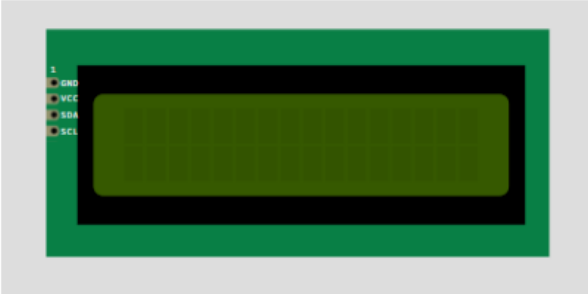


```
#include <Arduino.h>
#include <LiquidCrystal.h>

// Initialize the library with the numbers of the interface pins
// RS, E, D4, D5, D6, D7
LiquidCrystal lcd(19, 18, 17, 16, 4, 2);

void setup() {
  // Set up the LCD's number of columns and rows:
  lcd.begin(16, 2);

  // Print a message to the LCD.
  lcd.print("ESP32 Test...");

  delay(2000);
  lcd.clear();
}

void loop() {
  // Set the cursor to column 0, line 0
  lcd.setCursor(0, 0);
  lcd.print("System: Online");

  // Set the cursor to column 0, line 1
  lcd.setCursor(0, 1);
  lcd.print("Uptime: ");
  lcd.print(millis() / 1000);
  lcd.print("s");

  delay(1000);
}
```

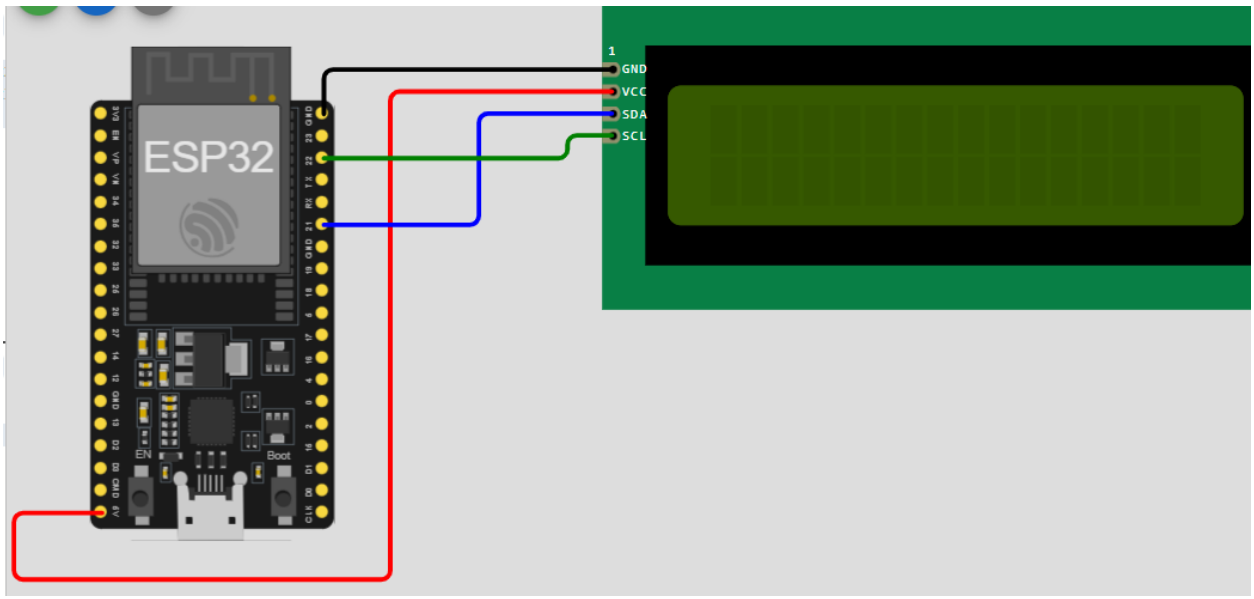


9. LCD 16x2 (I2C)

Same as above but uses only 2 wires (easier wiring).

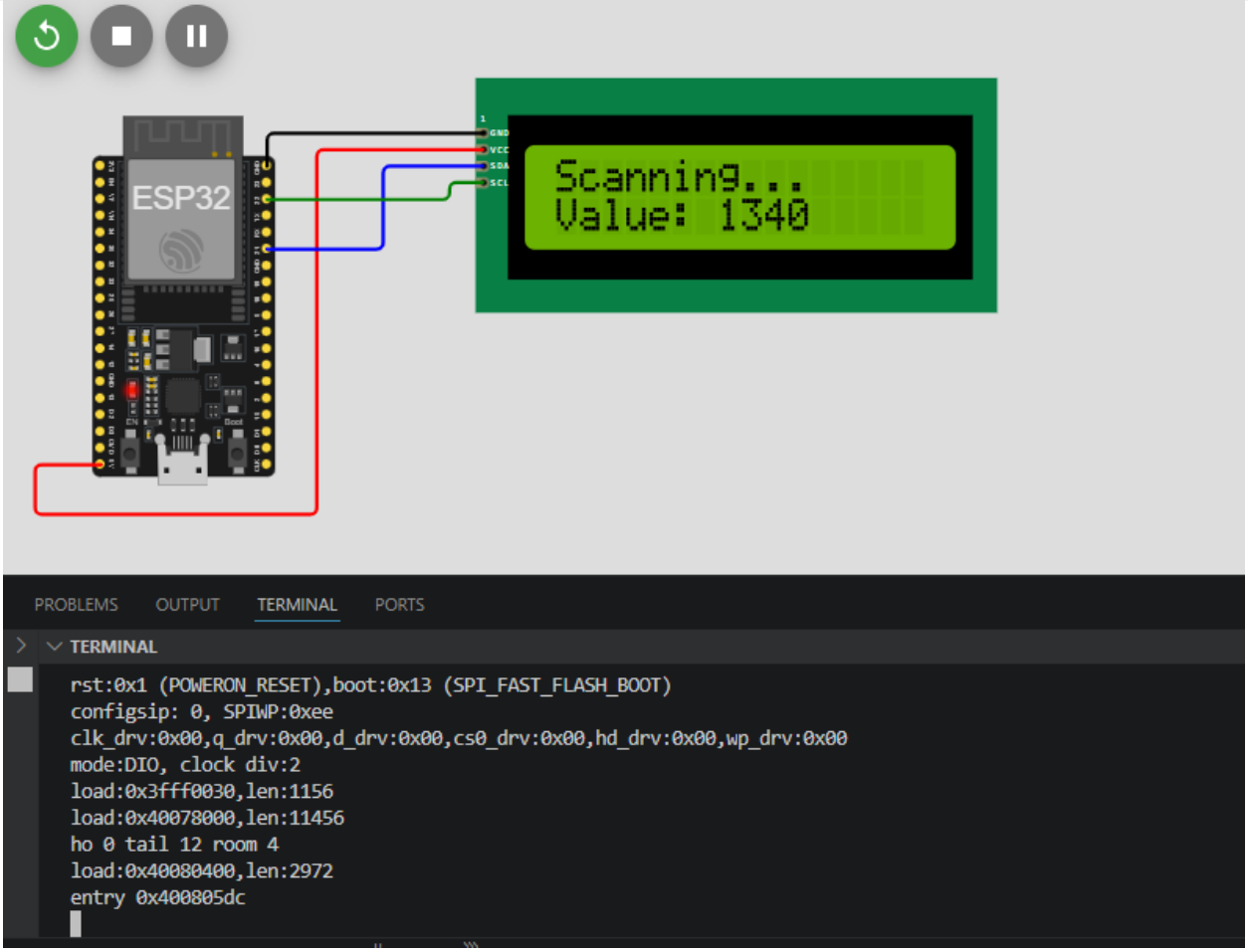


Pins of device	Pins of ESP32
GND	GND
VCC	5V
SDA	GPIO 22
SCL	GPIO 21




```
#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// 1. Set the LCD address to 0x27 for a 16 chars and 2 line display
// If 0x27 doesn't work, try 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  // 2. Initialize the LCD
  lcd.init();

  // Turn on the blacklight
  lcd.backlight();

  // Print a message
  lcd.setCursor(0, 0);
  lcd.print("I2C LCD Ready!");

  lcd.setCursor(0, 1);
  lcd.print("ESP32 Booting...");
  delay(2000);
  lcd.clear();
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Scanning...");
```

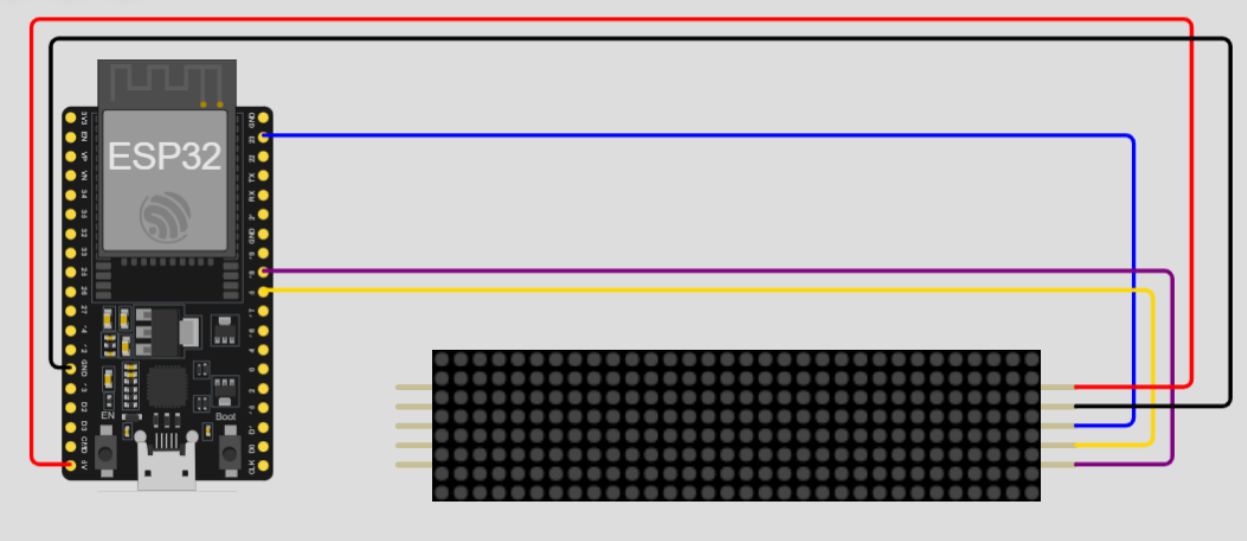
```
  // Example: Displaying a value
  int sensorValue = analogRead(34);
  lcd.setCursor(0, 1);
  lcd.print("Value: ");
  lcd.print(sensorValue);
  lcd.print("    "); // Clear trailing digits

  delay(500);
}
```



10. MAX7219 LED Dot Matrix

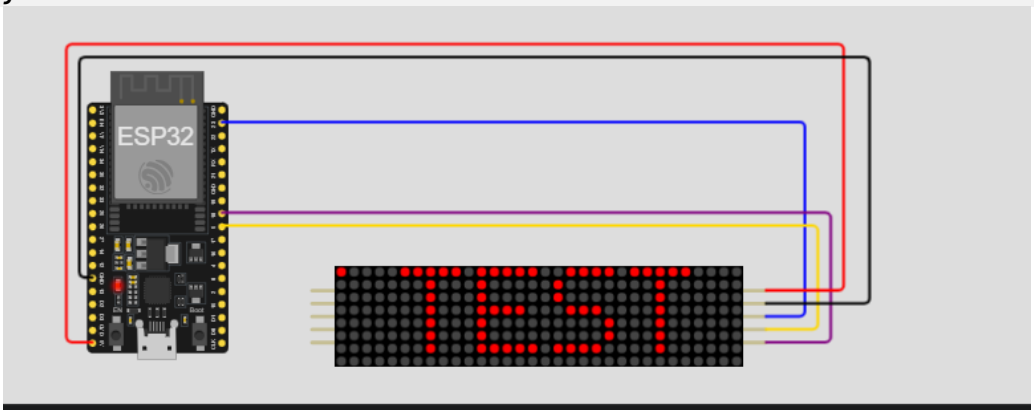
Displays scrolling text, animations, and patterns (8x8 matrix). Can daisy-chain multiple modules.

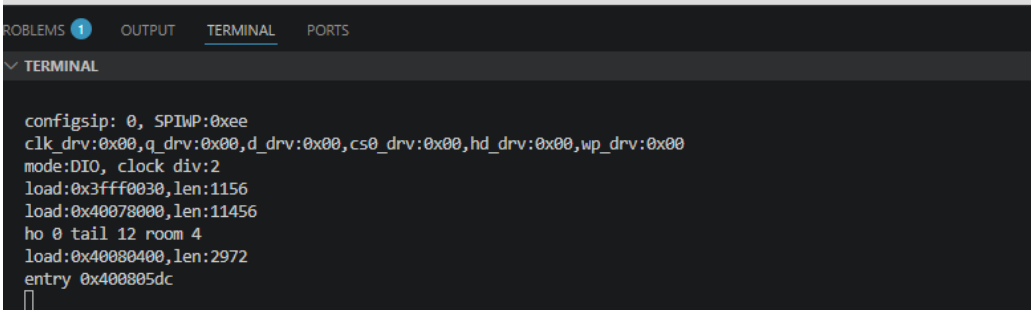
Pins of device	Pins of ESP32
V+	5V
GND	GND
DIN	GPIO 23
CS	GPIO 5
CLK	GPIO 18
V+.2	-
GND.2	-
DOUT	-
CS.2	-
CLK.2	-




```
#include <Arduino.h>
#include <MD_Parola.h>
#include <MD_MAX72xx.h>
#include <SPI.h>

// 1. Define Hardware Type and Pins
// Use the Parola hardware preset for the Wokwi MAX7219 matrix part
#define HARDWARE_TYPE MD_MAX72XX::PAROLA_HW
#define MAX_DEVICES 4 // Set this to the number of 8x8 matrices you have
#define CS_PIN 5
```

```
// 2. Initialize the Parola object
MD_Parola myDisplay = MD_Parola(HARDWARE_TYPE, CS_PIN, MAX_DEVICES);

void setup() {
  // 3. Initialize the display
  myDisplay.begin();

  // Set intensity (0-15)
  myDisplay.setIntensity(5);

  // Clear the display
  myDisplay.displayClear();

  // Configure the scrolling message once in setup
  myDisplay.displayText("ESP32 IoT TEST", PA_CENTER, 100, 0, PA_SCROLL_LEFT,
PA_SCROLL_LEFT);
  myDisplay.displayReset();
}

void loop() {
  // 4. Keep the configured animation running
  if (myDisplay.displayAnimate()) {
    myDisplay.displayReset();
  }
}
```

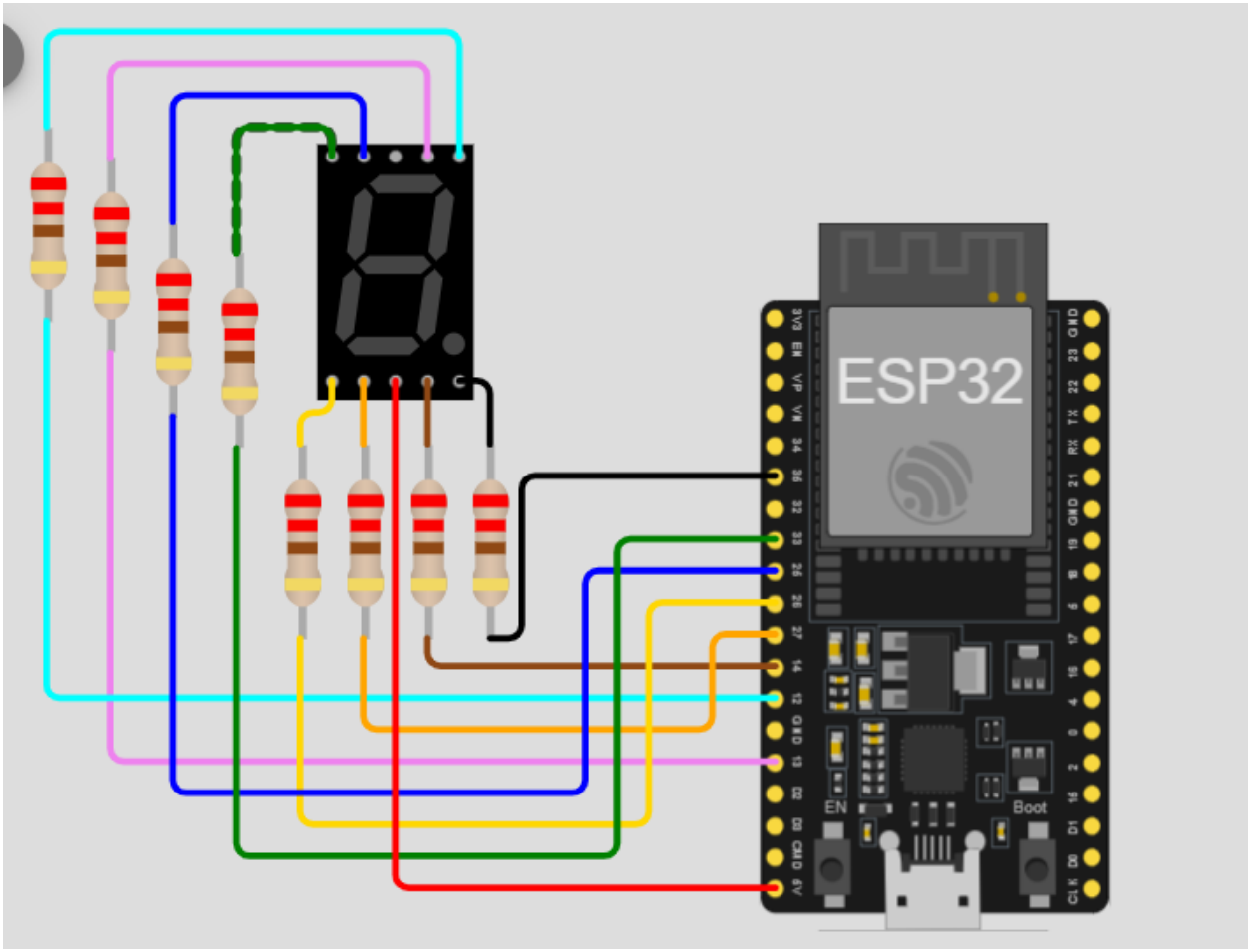




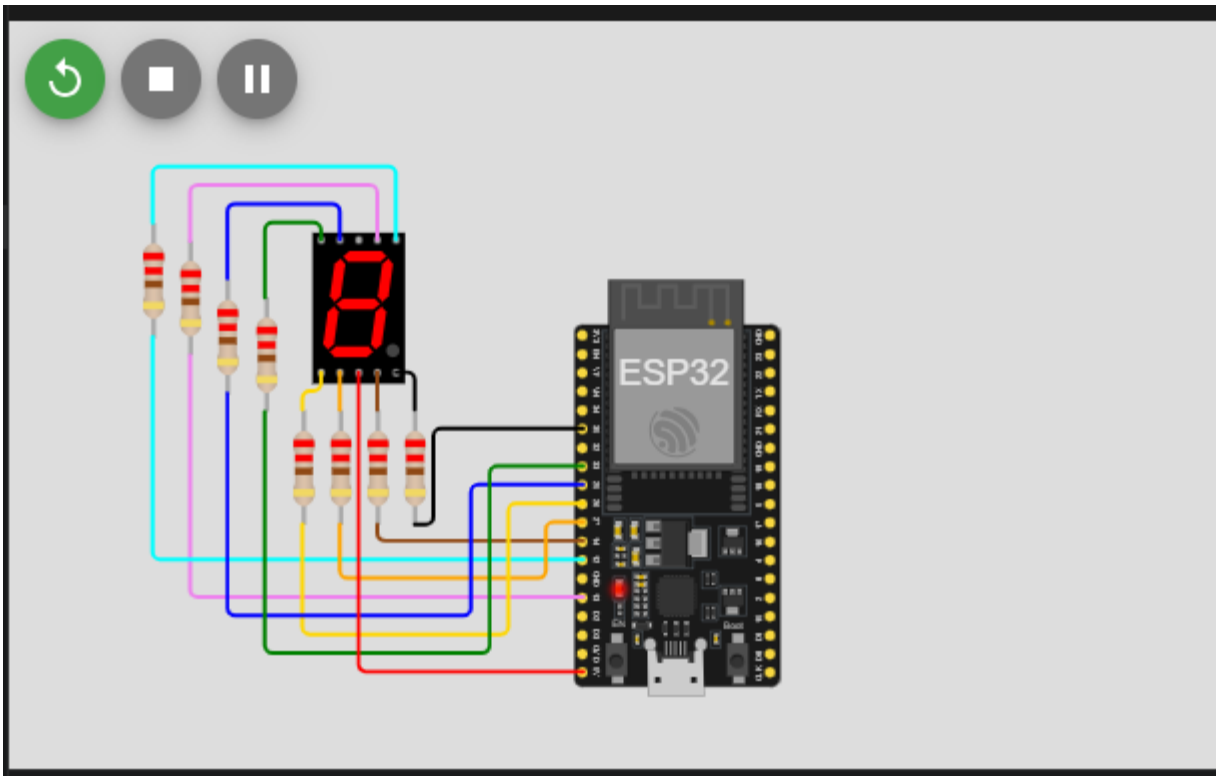
11. Seven Segment Display

Displays numbers and limited characters. Widely used in clocks, counters, and scoreboards.

Pins of device	Pins of ESP32
A	GPIO 13 (via 220Ω)
B	GPIO 12 (via 220Ω)
C	GPIO 14 (via 220Ω)
D	GPIO 27 (via 220Ω)
E	GPIO 26(via 220Ω)
F	GPIO 25 (via 220Ω)
G	GPIO 33 (via 220Ω)
DP	GPIO 32 (via 220Ω)
COM.1	5V
COM.2	-



```
#include <Arduino.h>
// Define pins for segments a, b, c, d, e, f, g
const int segments[7] = {13, 12, 14, 27, 26, 25, 33};

// Binary map for numbers 0-9 (a, b, c, d, e, f, g)
const byte digitMap[10] = {
  0b00111111, // 0
  0b00000110, // 1
  0b01011011, // 2
  0b01001111, // 3
  0b01100110, // 4
  0b01101101, // 5
  0b01111101, // 6
  0b00000111, // 7
  0b01111111, // 8
  0b01101111  // 9
};

void displayDigit(int num);

void setup() {
  for (int i = 0; i < 7; i++) {
    pinMode(segments[i], OUTPUT);
  }
}

void loop() {
  for (int i = 0; i <= 9; i++) {
    displayDigit(i);
    delay(1000);
  }
}

void displayDigit(int num) {
  for (int i = 0; i < 7; i++) {
    // Check the bit for each segment
    int bit = bitRead(digitMap[num], i);
    digitalWrite(segments[i], bit ? LOW : HIGH);
  }
}
```



PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
```

Default (17.LB04 Test17 Seven Segment Display) Auto Java: Ready